

UNIT 5

Memory Concepts and Hierarchy – Cache Memories: Mapping and Replacement Techniques – Virtual Memory – DMA – I/O – Accessing I/O: Parallel and Serial Interface – Interrupt I/O

Memory Concept

- Memories are made up of registers. Each register in the memory is one storage location also called memory location.
- Each memory location is identified by an address. The number of storage locations can vary from a few in some memories to hundreds of thousands in others. Each location can accommodate one or more bits.
- Generally, the total number of bits that a memory can store is its capacity. Most of the types the capacity is specified in terms of bytes (group of eight bits).
- Each register consists of storage elements (flip-flops or capacitors in semiconductor memories and magnetic domain in magnetic storage), each of which stores one bit of data. A storage element is called a cell.
- The data stored in a memory by a process called writing and are retrieved from the memory by a process called reading. Fig. 5.1.1 illustrates in a very simplified way the concept of write, read, address and storage capacity for a generalized memory.

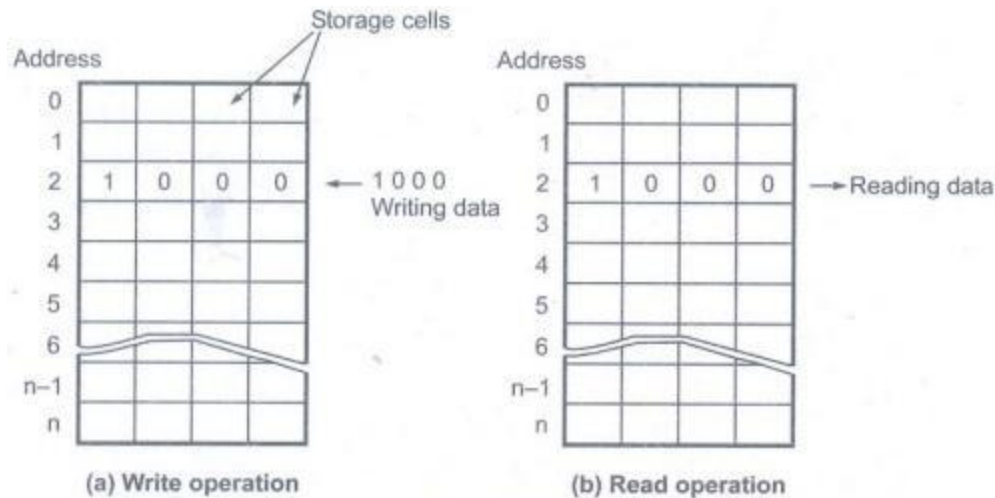


Fig. 8.1.1

- As shown in the Fig. 8.1.1 memory unit stores binary information in groups of bits called words. A word in memory is an entity of bits that moves in and out of storage as a unit. A word having group of 8-bits is called a byte. Most computer memories use words that are multiples of eight bits in length. Thus, a 16-bit word contains two bytes, and a 32-bit word is made of 4-bytes.

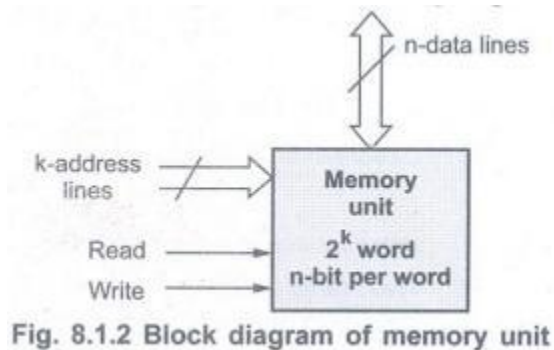


Fig. 8.1.2 Block diagram of memory unit

- The communication between a memory and its environment is achieved through data lines, address selection lines, and control lines that specify the direction of transfer.
- The Fig. 8.1.2 shows the block diagram of memory unit. Then data lines provide the information to be stored in memory and the k address lines specify the

particular word chosen among the many available. The two control inputs specify the direction transfer.

- When there are k address lines we can access 2^k memory words. For example, if $k = 10$ we can access $2^{10} = 1024$ memory words.

Illustrative Examples

Example 8.1.1 A bipolar RAM chip is arranged as 16 words. How many bits are stored in the chip?

Solution: $16 \times 8 = 128$ bits Therefore one word = 8 bits.

Example 8.1.2 How many address bits are needed to operate a $2\text{ K} \times 8$ ROM ?

Solution: 2 K memory locations = 2048 locations

Since $2^{11} = 2048$, we need 11 address lines.

Example 8.1.3 How many locations are addressed using 18 address bits?

Solution: The number of locations addressed = $2^{18} = 262144$

Characteristics of Memory

- The Table 8.1.1 lists the key characteristics of memory systems.

Location	CPU
	Internal (main)
	External (Secondary)
Capacity	Word size
	Number of words
Unit of transfer	Word
	Block
Access method	Sequential access
	Direct access
	Random access
	Associative access
Performance	Access time, Cycle time, Transfer rate
Physical type	Semiconductor
	Magnetic surface
Physical characteristics	Volatile / nonvolatile
Organisation	Erasable / nonerasable

Table 8.1.1

Physical characteristics:

Volatile/Nonvolatile If memory can hold data even if power is turned off, it is called nonvolatile memory; otherwise it is called volatile memory.

Erasable/Non-erasable The memories in which data is once programmed cannot be erased are called non-erasable memories. On the other hand, if data in the memory is erasable then memory is called erasable memory.

The Table 8.1.2 shows the characteristics of some common memory technologies.

Characteristics of some common memory technologies

Technology	Storage type	Access method	Alterability	Permanence	Typical access time t_A
Semiconductor RAM	Electronic	Random	Read/Write	Volatile	10 ns *
Semiconductor ROM	Electronic	Random	Read only	Non-volatile	10 ns
Magnetic (Hard) disk	Magnetic	Semirandom	Read/Write	Non-volatile	50 ns
Optical disk CD-ROM	Optical	Semirandom	Read only	Non-volatile	100 ms
Erasable optical disk	Optical	Semirandom	Read/Write	Non-volatile	100 ms
Magnetic tape	Magnetic	Serial	Read/Write	Non-volatile	1 s Depend on access location

Table 8.1.2

Distinguish between volatile and non-volatile memories

Sr. No.	Volatile memory	Non-volatile memory
1.	Volatile memory requires constant power to maintain the stored information.	Non-volatile memory can hold the stored information even when there is no power supply to the memory or storage device.
2.	Volatile memory is typically used only for primary storage.	Non-volatile memory is typically used for the task of secondary storage or long-term persistent storage.
3.	Type of volatile memories are : i) Random Access Memory (RAM) ii) Static RAM (SRAM) iii) Dynamic RAM (DRAM) iv) Fast Page Mode DRAM (FPM DRAM) v) Extended Data Output RAM (EDO RAM) vi) Synchronous DRAM (SDRAM) vii) Double Data Rate (DDR) SDRAM	Examples of non-volatile memory include read-only memory, flash memory, most types of magnetic computer storage devices (e.g. hard disks, floppy disk drives and magnetic tape), optical disc drives, and early computer storage methods such as paper type and punch cards.

- The processor of a computer can usually process instructions and data faster than they are fetched from the memory unit. The memory cycle time, then is the bottleneck in the system. One way to avoid this bottleneck is to use a cache

memory. Cache memory is a small, fast memory that is inserted between the larger, slower main memory and the processor. It usually holds the currently active segments of a program and their data.

- In most modern computers, the physical main memory is not as large as the address space spanned by an address issued by the processor. Here, the virtual memory technique is used to extend the apparent size of the physical memory. It uses secondary storage such as disks, to extend the apparent size of the physical memory.

Classification of Primary Memory

- The memory devices can be classified based on following parameters:
- Principle of operation
- Physical characteristics
- Mode of access and
- Terminology used for fabrication.
- The Fig. 8.1.3 shows the classification of memory.
- Broadly semiconductor memories are classified as volatile memories and non-volatile memories. Volatile memories can retain their state as long as power is applied. On the other hand, non-volatile memories can hold data even if power is turned off.
- Read/Write Memories (RWMs) are those memories, which allows both read and write operations. They are used in applications where data has to change

continuously. They are also used for temporary storage of data. ROM memories allow only read operation. They are used to store monitor programs and constants used in the program.

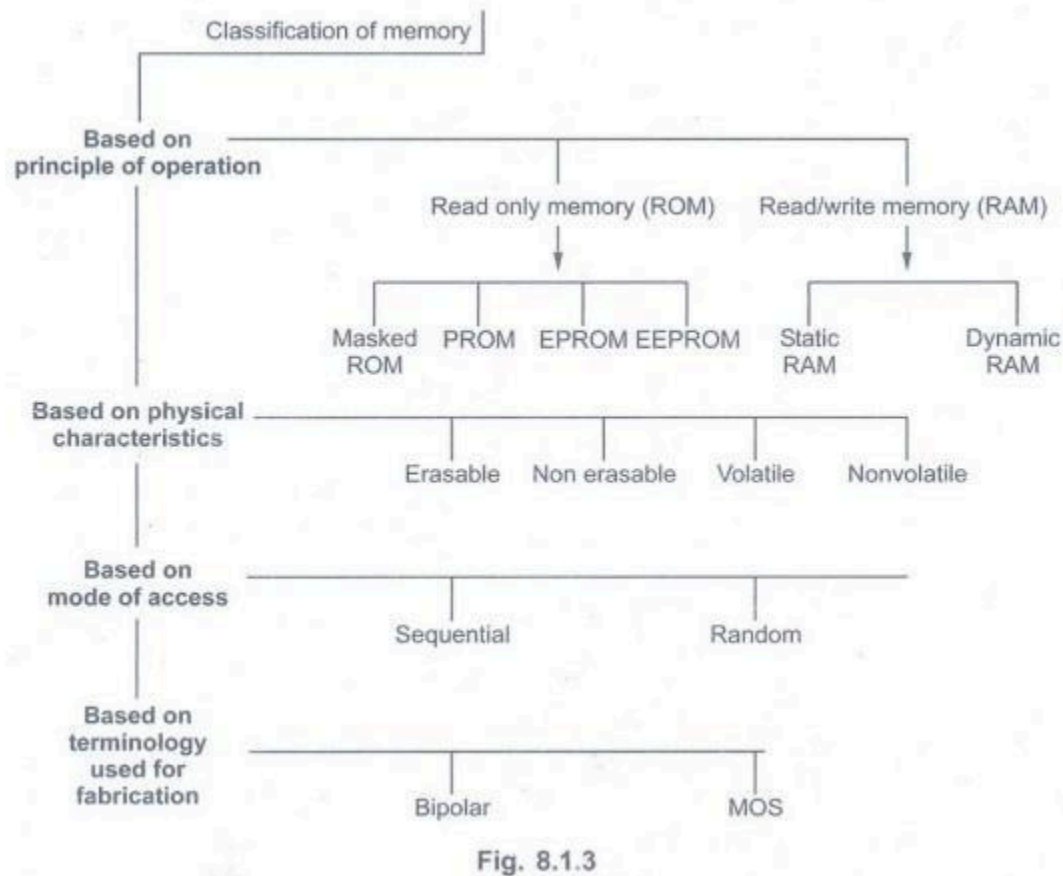


Fig. 8.1.3

- The volatile memories which can hold data as long as power is ON are called Static RAMS (SRAMs). Dynamic RAMS (DRAMs) stores the data as a charge on the capacitor and they need refreshing of charge on the capacitor after every few milliseconds to hold the data even if power is ON.
- EPROM and EEPROM are erasable memories in which the stored data can be erased and new data can be stored.

- The semiconductor memories are also classified as Bipolar and MOS memories depending upon the type of transistors used to construct the individual cell.

Classification of Secondary Memory

- A primary memory is costly and has a limited size. This memory is mainly used for storing the currently processing data.
- Secondary storage is used to store data and instructions (programs) when they are not being processed.
- The devices those are used as secondary storage are non-volatile and have a larger storage capacity. Also, they are less expensive as compared to primary storage devices.
- However, they are slower in comparison. The examples are hard disks, floppies, CD-ROMs, magnetic tapes etc. This type of memory is also called secondary memory, auxiliary memory or peripheral storage.
- Fig. 8.1.4 shows the classification of secondary storage devices. They can be categorized broadly according to their access types as sequential and random (direct).

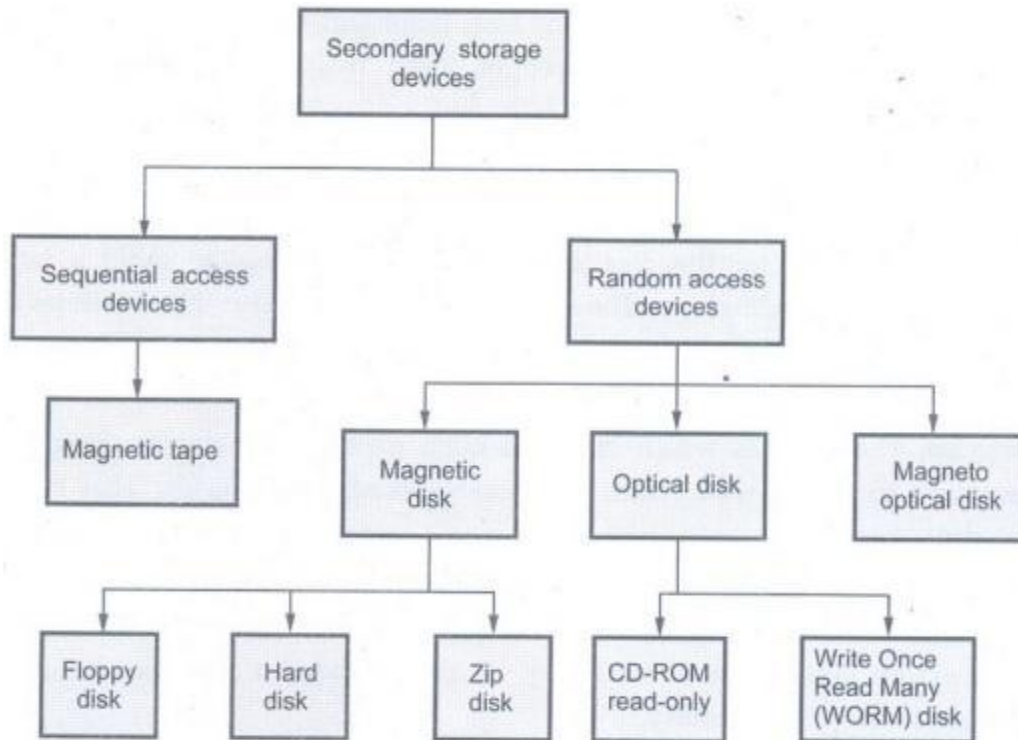


Fig. 8.1.4 Classification of secondary storage devices

Memory Hierarchy AU; Dec.-14, 18, May-19

- Ideally, computer memory should be fast, large and inexpensive. Unfortunately, it is impossible to meet all the three of these requirements using one type of memory.
- Increased speed and size are achieved at increased cost.
- Very fast memory system can be achieved if SRAM chips are used. These chips are expensive and for the cost reason it is impracticable to build a large main memory using SRAM chips. The only alternative is to use DRAM chips for large main memories.

- Processor fetches the code and data from the main memory to execute the program. The DRAMS which form the main memory are slower devices. So it is necessary to insert wait states in memory read/write cycles. This reduces the speed of execution.
- The solution for this problem is come out with the fact that most of the computer programs work with only small sections of code and data at a particular time. In the memory system, small section of SRAM is added along with main memory, referred to as cache memory.
- The program which is to be executed is loaded in the main memory, but the part of program (code) and data that work at a particular time is usually accessed from the cache memory.
- This is accomplished by loading the active part of code and data from main memory to cache memory. The cache controller looks after this swapping between main memory and cache memory with the help of DMA controller.
- The cache memory just discussed is called secondary cache. Recent processors have the built-in cache memory called primary cache.
- DRAMs along with cache allow main memories in the range of tens of megabytes to be implemented at a reasonable cost, the size and better speed performance. But the size of memory is still small compared to the demands of large programs with voluminous data. A solution is provided by using secondary storage, mainly magnetic disks and magnetic tapes to implement large memory spaces. Very large disks are available at a reasonable price, sacrificing the speed.

- From the above discussion, we can realize that to make efficient computer system it is not possible to rely on a single memory component, but to employ a memory hierarchy. Using memory hierarchy all of different types of memory units are employed to give efficient computer system. A typical memory hierarchy is in Fig. 8.2.1.

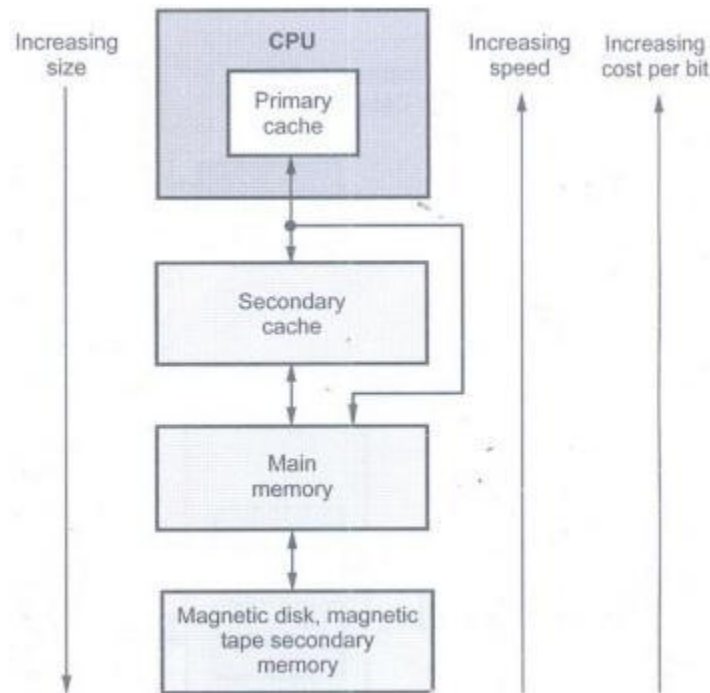


Fig. 8.2.1

Memory hierarchy

- In summary, we can say that a huge amount of cost-effective storage can be provided by magnetic disks. A large, yet affordable, main memory can be built with DRAM technology along with the cache memory to achieve better speed performance.

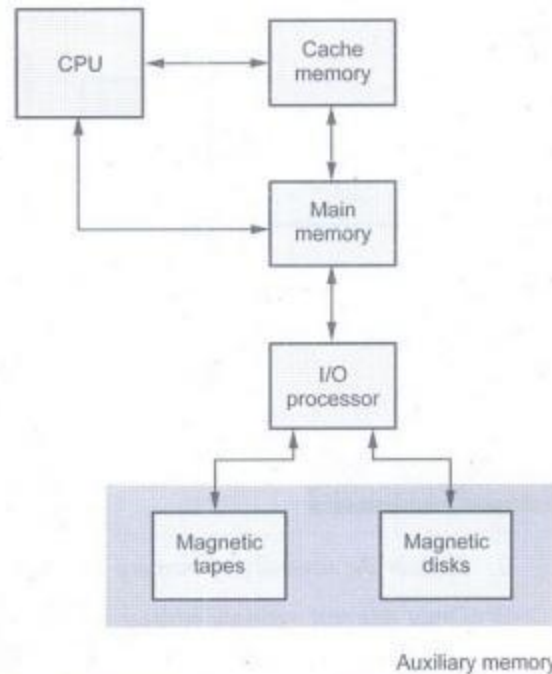
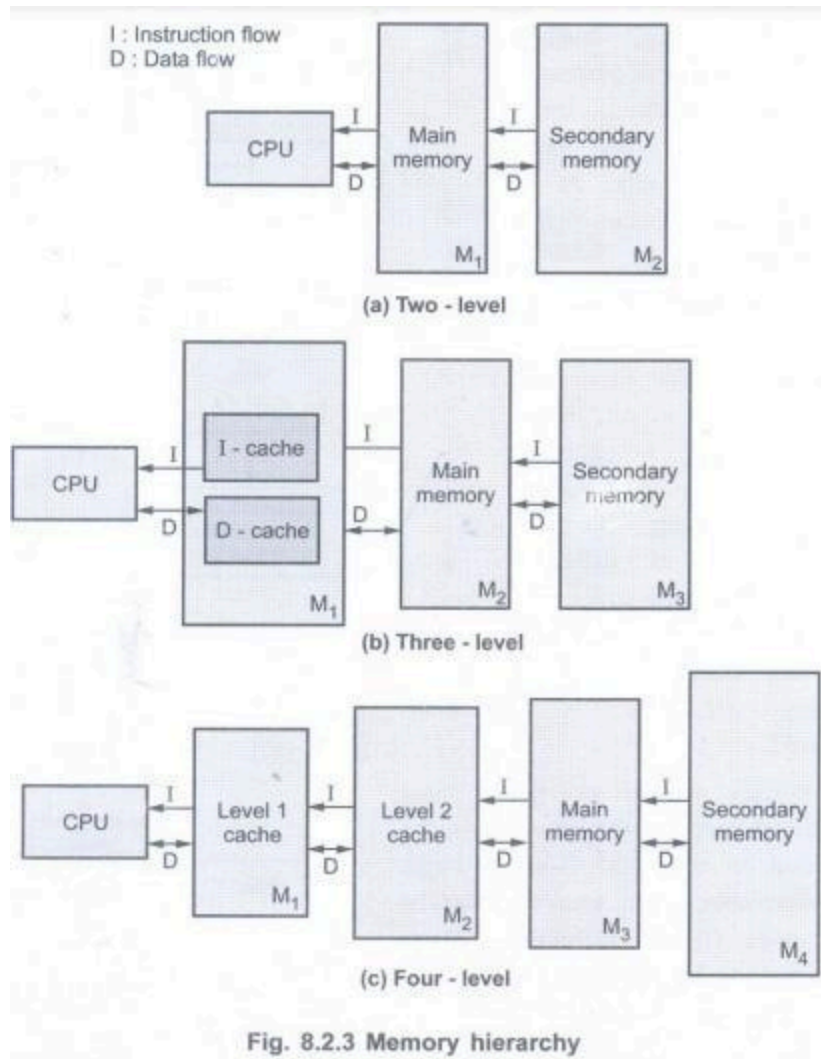


Fig. 8.2.2 Memory hierarchy in a computer system

- The Fig. 8.2.2 shows how memory hierarchy can be employed in a computer system. As shown in the Fig. 8.2.2, the bottom of the hierarchy magnetic tapes and magnetic disks are used as a secondary memory. This memory is also known auxiliary memory.
- The main memory occupies a central position by being able to communicate directly with the CPU and with auxiliary memory devices through an I/O processor.

Fig. 8.2.3 shows common memory hierarchies with two, three and four levels.



Cache Memories

AU: Dec.-06,07,08,09,11,12,14,17,18, May-08,09,11,12,13,16,17,19, June-08,13

- In a computer system the program which is to be executed is loaded in the main memory (DRAM). Processor then fetches the code and data from the main memory to execute the program. The DRAMS which form the main memory are slower

devices. So it is necessary to insert wait states in memory read/write cycles. This reduces the speed of execution.

- To speed up the process, high speed memories such as SRAMS must be used. But considering the cost and space required for SRAMS, it is not desirable to use SRAMS to form the main memory. The solution for this problem is come out with the fact that most of the microcomputer programs work with only small sections of code and data at a particular time.

- **Definition:** The part of program (code) and data that work at a particular time is usually accessed from the SRAM memory. This is accomplished by loading the active part of code and data from main memory to SRAM memory. This small section of SRAM memory added between processor and main memory to speed up execution process is known as cache memory.

- Fig. 8.4.1 shows a simplest form of cache memory system.

- A cache memory system includes a small amount of fast memory () and a large amount of slow memory (DRAM). This system is configured to simulate a large amount of fast memory.

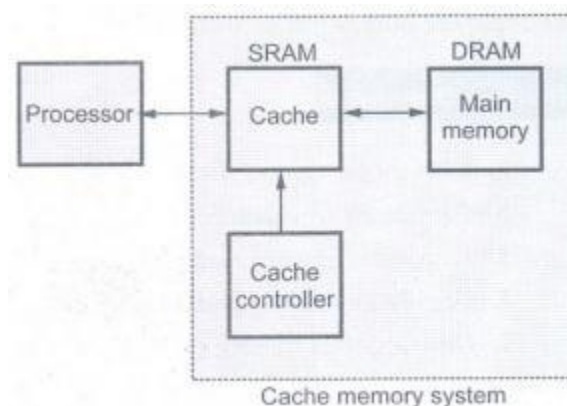


Fig. 8.4.1 Cache memory system

- Cache controller implements the cache logic. If processor finds that the addressed code or data is not available in cache - the condition referred to as cache miss, the desired memory block is copied from main memory to cache using cache controller. The cache controller decides which memory block should be moved in or out of the cache and in or out of main memory, based on the requirements. (The cache block is also known as cache slot or cache line.)
- The percentage of accesses where the processor finds the code or data word it needs in the cache memory is called the hit rate/hit ratio. The hit rate is normally greater than 90 percent.

Hit rate = $\text{Number of hits} / \text{Total number of bus cycles} \times 100 \%$

Example 8.4.1 The application program in a computer system with cache uses 1400 instruction acquisition bus cycle from cache memory and 100 from main memory. What is the hit rate? If the cache memory operates with zero wait state and the main memory bus cycles use three wait states, what is the average number of wait states experienced during the program execution ?

Solution : Hit rate = $1400 / (1400 + 100) \times 100 = 93.3333 \%$

Total wait states = $1400 \times 0 + 100 \times 3 = 300$

Average wait states = $\text{Total wait states} / \text{Number of memory bus cycles}$

= $300 / 1500 = 0.2$

Most Commonly used Cache Organizations

- Two most commonly used system organizations for cache memory are :

- Look-aside and
- Look-through

Look-aside system organization

- The Fig. 8.4.2 shows system of look-aside cache organization. Here, the cache and the main memory are directly connected to the system bus.

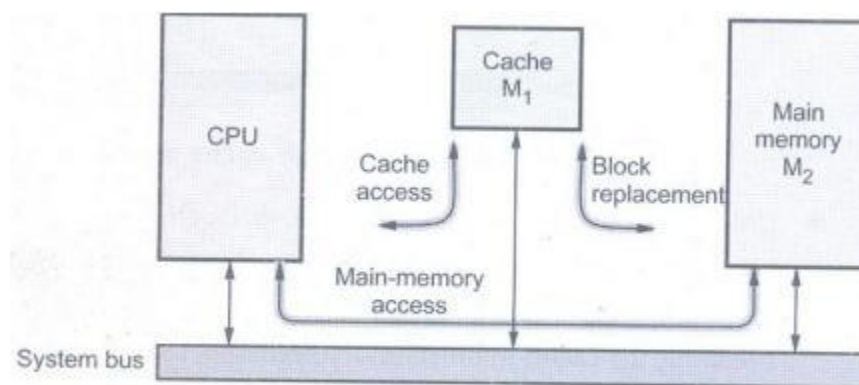


Fig. 8.4.2 Look-aside cache system organization

- In this system, the CPU initiates a memory access by placing a physical address on the memory address bus at the start of read or write cycle.
- The cache memory M_1 immediately compares physical address to the tag addresses currently residing in its tag memory. If a match is found, i.e., in case of cache hit, the access is completed by a read or write operation executed in the cache. The main memory is not involved in the process of read or writes.
- If match is not found, i.e., in case of cache miss, the desired access is completed by a read or write operation directed to M_2 . In response to cache miss, a block of data that includes the target address is transferred from M_2 to M_1 . The system bus is used for this transfer and hence it is unavailable for other uses like I/O operations.

Look-through system organization

- The Fig. 8.4.3 shows look-through system of cache organization. Here, the CPU communicates with cache via a separate (local) bus which is isolated from the main system bus. Thus during cache accesses, the system bus is available for use by other units, such as I/O controllers, to communicate with main memory.

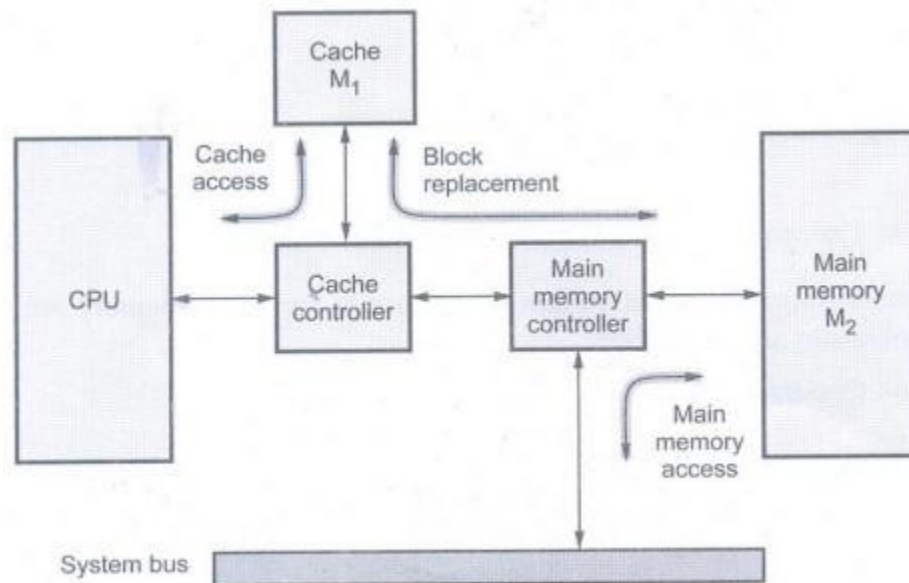


Fig. 8.4.3 Look-through cache system organization

- Unlike the look-aside system, look-through cache system does not automatically send all memory requests to main memory; it does so only after a cache miss.
- A look-through cache systems use wider local bus to link M_1 and M_2 , thus speeding up cache-main-memory transfers (block transfers).
- Look-through cache system is faster.

Disadvantages:

- It is complex.

- It is costly.
- It takes longer time for M_2 to respond to the CPU when a cache miss occurs.

Cache Read Operation

- The Fig. 8.4.4 shows the small cache system. Here each cache block is 4 bytes and each memory address is 10-bit long. Due to this 8 high-order bits form the tag or block address and the 2 low-order bits define a displacement address within the block.

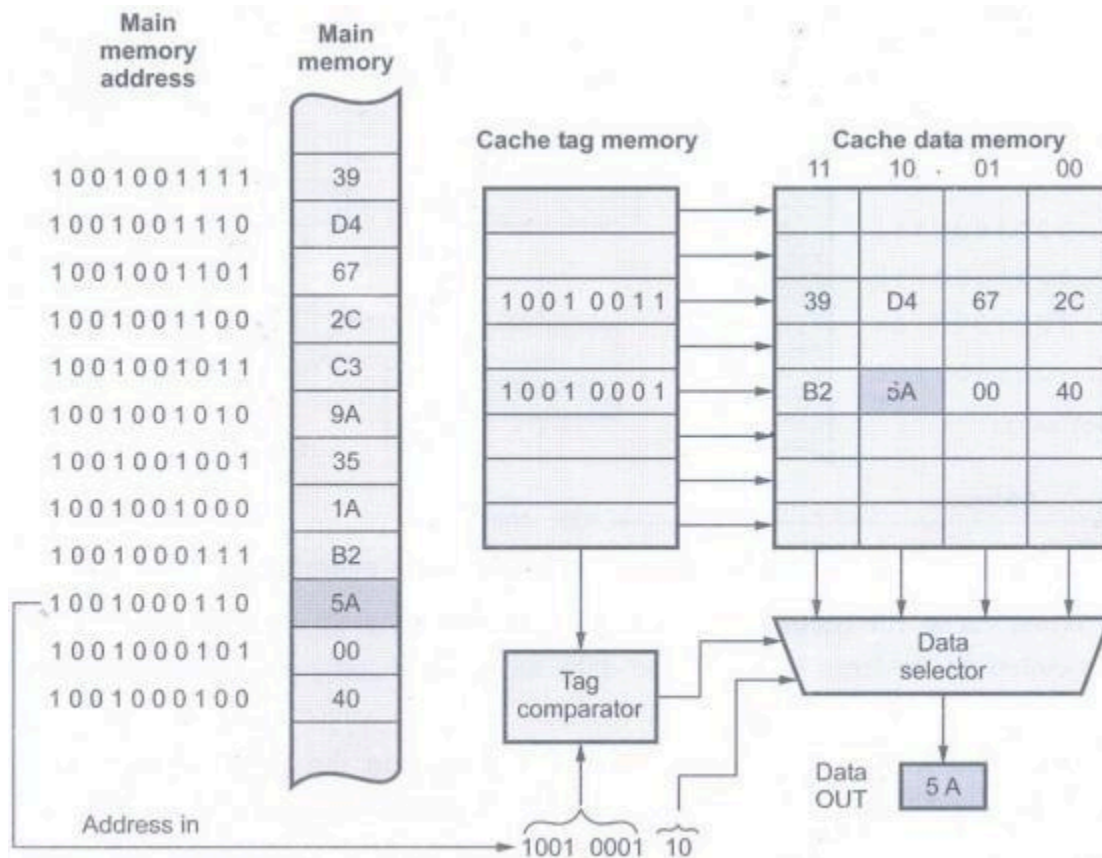


Fig. 8.4.4 Execution of cache read operation

- When a block is assigned to the cache data memory, its tag is also placed in the cache tag memory.
- During read operation, the 8 high-order bits of an address are compared with stored tags in the cache tag memory to find match (cache hit). The stored tag pinpoints the corresponding block in cache data memory and the 2-bit displacement is used to read the target word.

Cache Write Operation

- The Fig. 8.4.5 shows execution of cache write operation. It uses same addressing technique as in case of read operation.

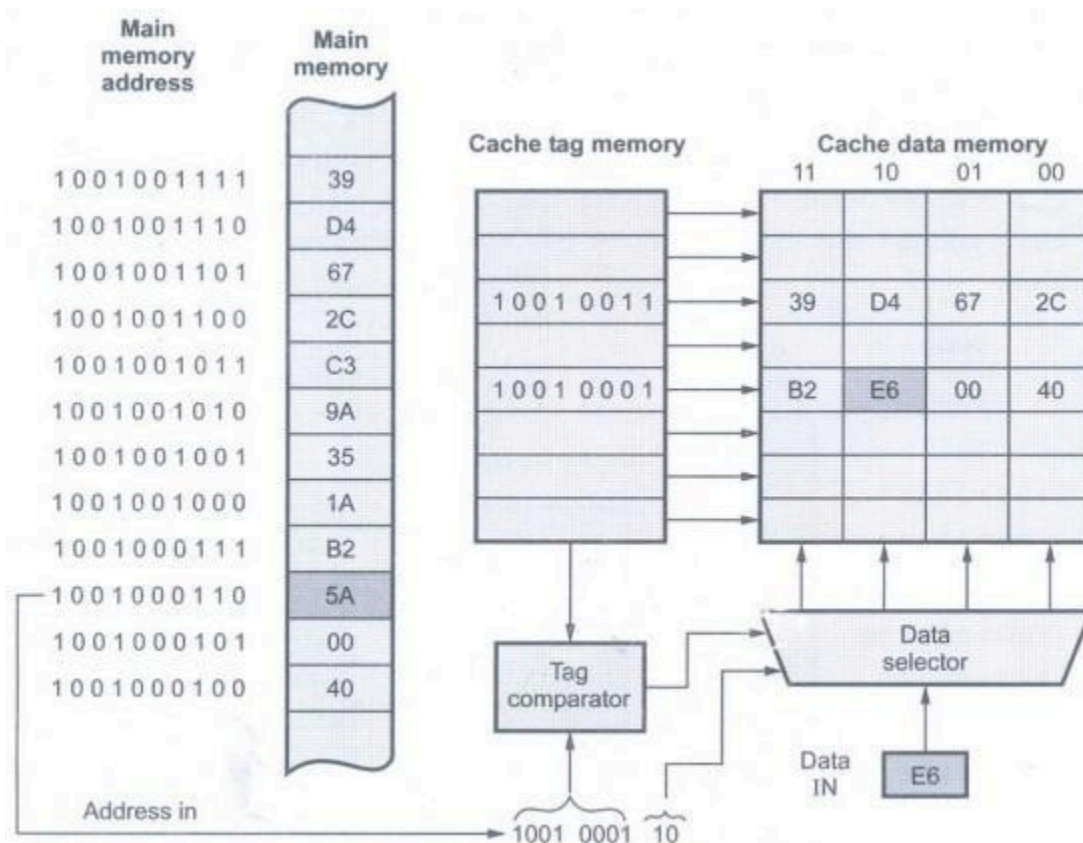


Fig. 8.4.5 Execution of cache write operation

- When cache hit occurs, the new data, in this case E6, is stored at the location pointed by address in the cache data memory, thereby overwriting the old data 5 A.
- Now data in the cache data memory and data in the main memory for given address is different. This causes cache consistency problem.

Program Locality

- In cache memory system, prediction of memory location for the next access is essential. This is possible because computer systems usually access memory from the consecutive locations. This prediction of next memory address from the current memory address is known as program locality.
- Program locality enables cache controller to get a block of memory instead of getting just a single address.
- The principle of program locality may not work properly when program executes JUMP and CALL instructions. In case of these instructions, program code is not in sequence.

Locality of Reference

- We know that program may contain a simple loop, nested loops or a few procedures that repeatedly call each other. The point is that many instructions in localized area of the program are executed repeatedly during some time period and the remainder of the program is accessed relatively infrequently. This is referred to as locality of reference.
- It manifests itself in two ways: temporal and spatial.

- The temporal means that a recently executed instruction is likely to be executed again very soon.
- The spatial means that instructions stored nearby to the recently executed instruction are also likely to be executed soon.
- The temporal aspect of the locality of reference suggests that whenever an instruction or data is first needed, it should be brought into the cache and it should remain there until it is needed again.
- The spatial aspect suggests that instead of bringing just one instruction or data from the main memory to the cache, it is wise to bring several instructions and data items that reside at adjacent address as well. We use the term block to refer to a set of contiguous addresses of some size.

Block Fetch

- Block fetch technique is used to increase the hit rate of cache.
- A block fetch can retrieve the data located before the requested byte (look behind) or data located after the requested byte (look ahead) or both.
- When CPU needs to access any byte from the block, entire block that contains the needed byte is copied from main memory into cache.
- The size of the block is one of the most important parameters in the design of a cache memory system.

Block size

1. If the block size is too small, the look-ahead and look-behind are reduced and therefore the hit rate is reduced.
2. Larger blocks reduce the number of blocks that fit into a cache. As the number of blocks decrease, block rewrites from main memory becomes more likely.
3. Due to large size of block, the ratio of required data and useless data is less.
4. Bus size between the cache and the main memory increases with block size to accommodate larger data transfers between main memory and the cache, which increases the cost of cache memory system.

Elements of Cache Design

- The cache design elements include cache size, mapping function, replacement algorithm write policy, block size and number of caches.

Cache size: The size of the cache should be small enough so that the overall average cost per bit is close to that of main memory alone and large enough so that the overall average access time is close to that of the cache alone.

Mapping function: The cache memory can store a reasonable number of blocks at any given time, but this number is small compared to the total number of blocks in the main memory. Thus we have to use mapping functions to relate the main memory blocks and cache blocks. There are two mapping functions commonly used direct mapping and associative mapping. Detail description of mapping functions is given in section 8.4.6.

Replacement algorithm: When a new block is brought into the cache, one of the existing blocks must be replaced, by a new block.

- There are four most common replacement algorithms:
- Least-Recently-Used (LRU)
- First-In-First-Out (FIFO)
- Least-Frequently-Used (LFU)
- Random
- Cache design change according to the choice of replacement algorithm. Detail description of replacement algorithm is given in section 8.4.8.

Write policy: It is also known as cache updating policy. In cache system, two copies of the same data can exist at a time, one in cache and one in main memory. If one copy is altered and other is not, two different sets of data become associated with the same address. To prevent this, the cache system has updating systems such as: write through system, buffered write through system and write-back system. The choice of cache write policy also changes the design of cache.

Block size: It should be optimum for cache memory system.

Number of caches: When on-chip cache is insufficient, the secondary cache is used. The cache design changes as number of caches used in the system changes.

Mapping

- Usually, the cache memory can store a reasonable number of memory blocks at any given time, but this number is small compared to the total number of blocks in the main memory. The correspondence between the main memory blocks and those in the cache is specified by a mapping function.

- The mapping techniques are classified as:

1. Direct-mapping technique
2. Associative-mapping technique (Fully-associative)
3. Set-associative techniques.

To discuss these techniques of cache mapping we consider a cache consists of 128 blocks of 16 words each, for a total of 2048 (2 K) words and assume that the main memory has 64 K words. This 64 K words of main memory is addressable by a 16-bit address and it can be viewed as 4 K blocks of 16 words each. The groups of 128 blocks of 16 words each in main memory form a page.

1. Direct-Mapping

- It is the simplest mapping technique.
- In this technique, each block from the main memory has only one possible location in the cache organization. In this example, the block i of the main memory maps onto block j ($j = i \text{ modulo } 128$) of the cache, as shown in Fig. 8.4.6. Therefore, whenever one of the main memory blocks 0, 128, 256 ... is loaded in are stored in cache, it is stored in cache block 0. Blocks 1, 129, 257 ... block 1 and so on.

In general the mapping expression is

$$j = i \text{ modulo } m$$

where

i = Main memory block number

j = Cache block (line) number

m = Number of blocks (lines) in the cache

- To implement such cache system, the address is divided into three fields, as shown in Fig. 8.4.6.
- The lower order 4-bits select one of the 16 words in a block. This field is known as word field.
- The second field known as block field is used to distinguish a block from other blocks. Its length is 7-bits since $2^7 = 128$.
- When a new block enters the cache, the 7-bit cache block field determines the cache position in which this block must be stored.
- The third field is a tag field. It is used to store the high-order 5-bits of memory address of the block. These 5-bit (tag bits) are used to identify which of the 32 blocks (pages) that are mapped into the cache.

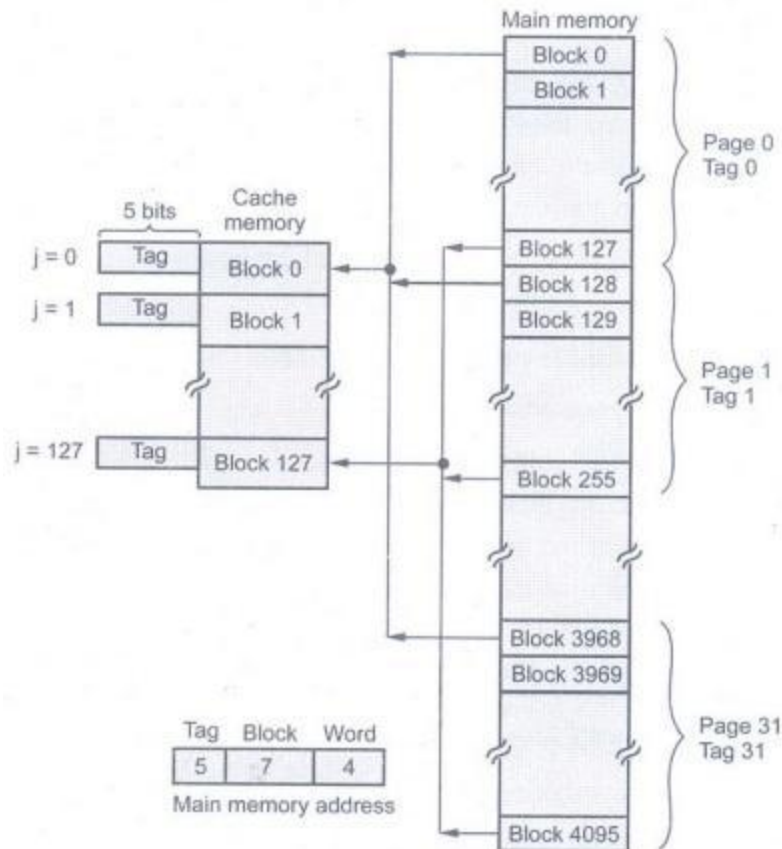


Fig. 8.4.6 Direct-mapped cache

- When memory is accessed, the 7-bit cache block field of each address generated by CPU points to a particular block location in the cache. The high-order 5-bits of the address are compared with the tag bits associated with that cache location. If they match, then the desired word is in that block of the cache. If there is no match, then the block containing the required word must first be read from the main memory and loaded into the cache.
- This means that to determine whether requested word is in the cache, only tag field is necessary to be compared. This needs only one comparison.
- The main drawback of direct mapped cache is that if processor needs to access same memory locations from two different pages of the main memory frequently,

the controller has to access main memory frequently. Since only one of these locations can be in the cache at a time. For example, if processor want to access memory location 100H from page 0 and then from page 2, the cache controller has to access page 2 of the main memory. Therefore, we can say that direct-mapped cache is easy to implement, however, it is not very flexible.

2. Associative-Mapping (Fully-Associative Mapping)

- The Fig. 8.4.7 shows the associative-mapping technique. In this technique, a main memory block can be placed into any cache block position. As there is no fix block, the memory address has only two fields: word and tag. This technique is also referred to as fully-associative cache.
- The 12-tag bits are required to identify a memory block when it is resident in the cache. The high-order 12-bits of an address received from the CPU are compared to the tag bits of each block of the cache to see if the desired block is present.

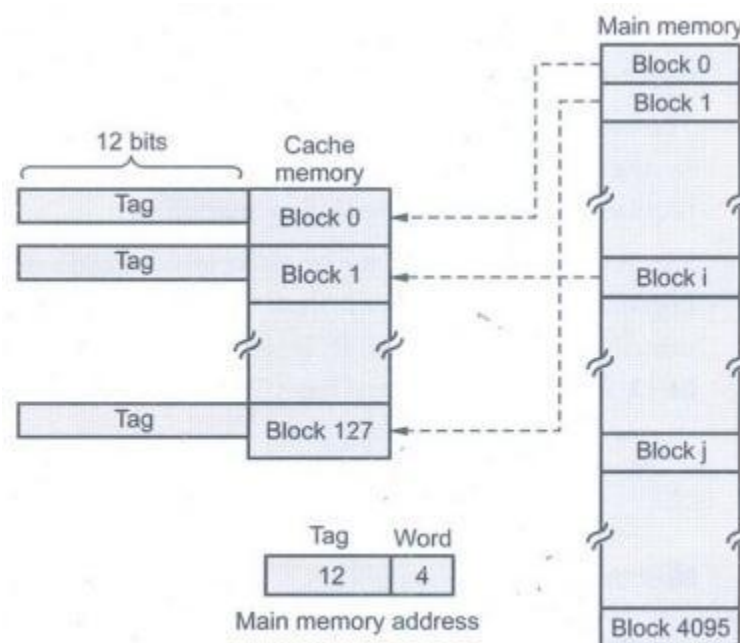


Fig. 8.4.7 Associative-mapped cache

- Once the desired block is present, the 4-bit word is used to identify the necessary word from the cache.
- This technique gives complete freedom in choosing the cache location in which to place the memory block. Thus, the memory space in the cache can be used more efficiently.
- A new block that has to be loaded into the cache has to replace (remove) an existing block only if the cache is full.
- In such situations, it is necessary to use one of the possible replacement algorithms to select the block to be replaced.
- **Disadvantage:** In associative-mapped cache, it is necessary to compare the higher-order bits of address of the main memory with all 128 tag corresponding to each block to determine whether a given block is in the cache. This is the main disadvantage of associative-mapped cache.

3. Set-Associative Mapping

- The set-associative mapping is a both direct and associative mapping.
- It contains several groups of direct-mapped blocks that operate as several direct-mapped caches in parallel.
- A block of data from any page in the main memory can go into a particular block location of any direct-mapped cache. Hence the contention problem of the direct-mapped technique is eased by having a few choices for block placement.

- The required address comparisons depend on the number of direct-mapped caches in the cache system. These comparisons are always less than the comparisons required in the fully-associative mapping.
- Fig. 8.4.8 shows two way set-associative cache. Each page in the main memory is organized in such a way that the size of each page is same as the size of one directly mapped cache. It is called two-way set-associative cache because each block from main memory has two choices for block placement.

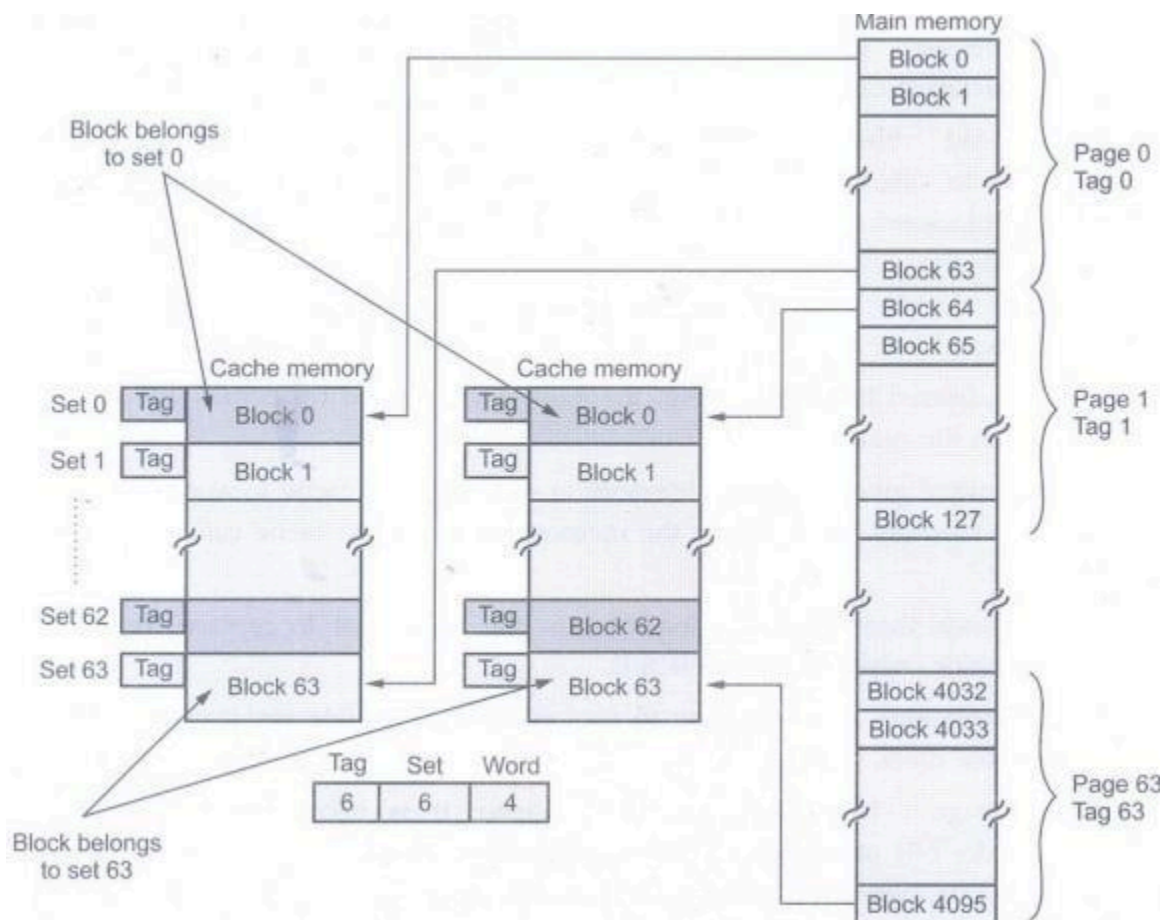


Fig. 8.4.8 Two-way set-associative cache

Two way set-associative cache

- In this technique, block 0, 64, 128,.....4032 of main memory can map into any of the two (block 0) blocks of set 0, block 1, 65, 129,,....., 4033 of main memory can map into any of the two (block 1) blocks of set 1 and so on.
- As there are two choices, it is necessary to compare address of memory with the tag bits of corresponding two block locations of particular set. Thus for two-way set-associative cache, we require two comparisons to determine whether a given block is in the cache.
- Since there are two direct-mapped caches, any two bytes having same offset from different pages can be in the cache at a time. This improves the hit rate of the cache system.
- To implement set-associative cache system, the address is divided into three fields, as shown in Fig. 8.4.8.
- The 4-bit word field selects one of the 16 words in a block.
- The set field needs 6-bits to determine the desired block from 64 sets. However, there are now 64 pages. To identify a block belongs to a particular page from 64 pages, six tag bits are required.

Example 8.4.2 Consider a cache consisting of 256 blocks of 16 words each, for a total of 4096 (4 K) words and assume that the main memory is addressable by a 16-bit address and it consists of 4 K blocks. How many bits are there in each of the TAG, BLOCK/SET and word fields for different mapping techniques ?

Solution: We know that memory address is divided into three fields. We will now find the exact bits required for each field in different mapping techniques.

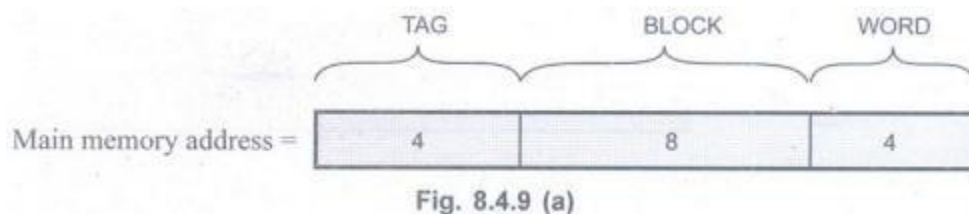
a) Direct-mapping

Word bits: We know that each block consists of 16 words. Therefore, to identify each word we must have ($2^4 = 16$) four bit reserved for it.

Block bits: The cache memory consists of 256 blocks and using direct-mapped technique, block k of the main memory maps onto block k modulo 256 of the cache. It has one to one correspondence and requires unique address for each block. To address 128 block we require ($2^8 = 256$) eight bits.

Tag bits: The remaining 4 ($16 - 4 - 8$) address bits are tag bits which stores the higher address of the main memory.

The main memory address for direct-mapping technique is divided as shown below:



b) Associative-mapping

Word bits: The word length will remain same i.e. 4 bits.

In the associative-mapping technique, each block in the main memory is identified by the tag bits and an address received from the CPU is compared with the tag bits of each block of the cache to see if the desired block is present. Therefore, this type of technique does not have block bits, but all remaining bits (except word bits) are reserved as tag bits.

Block bits: 0

Tag bits: To address each block in the main memory ($2^{12} = 4096$) 12 bits are required and therefore, there are 12 tag bits.

The main memory address for direct mapping technique is divided as shown below:

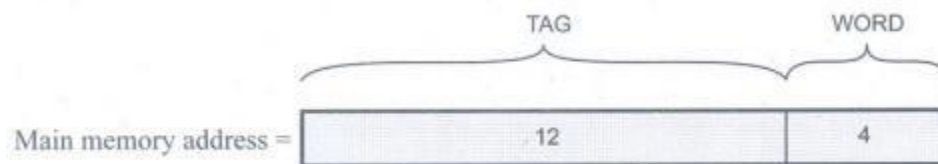


Fig. 8.4.9 (b)

c) Set-associative mapping

Let us assume that there is a 2-way set-associative mapping. Here, cache memory is mapped with two blocks per set. The set field of the address determines which set of the cache might contain the desired block.

Word bits: The word length will remain same i.e. 4 bits.

Set bits: There are 128 sets ($256/2$). To identify each set ($2^7 = 128$) seven bits are required.

Tag bits: The remaining 5 ($16 - 4 - 7$) address bits are the tag bits which stores higher address of the main memory.

The main memory address for 2-way set associative mapping technique is divided as shown below:

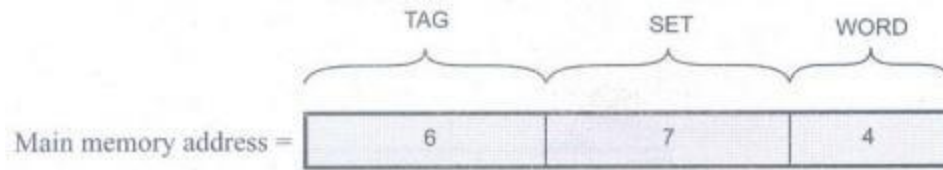


Fig. 8.4.10

Example 8.4.3 A block set-associative cache consists of 64 blocks divided into 4 block sets. The main memory contains 4096 blocks, each consists of 128 words of 16 bits length:

- i) How many bits are there in main memory?
- ii) How many bits are there in each of the TAG, SET and WORD fields?

Solution: i) Number of bits in main memory:

= Number of blocks x Number of words per block x Number of bits per word

$$= 4096 \times 128 \times 16$$

$$= 8388608 \text{ bits}$$

ii) Number of bits in word field:

There are 128 words in each block. Therefore, to identify each word ($2^7 = 128$) 7 bits are required.

iii) Number of T bits in set field: There are 64 blocks and each set consists of 4 blocks.

Therefore, there are 16 ($64/4$) sets. To identify each set ($2^4 = 16$) four bits are required.

iv) Number of bits in tag field: The total words in the memory are :

$$4096 \times 128 = 524288$$

To address these words we require ($2^{19} = 524288$) 19 address lines. Therefore, tag bits are eight ($19-7-4$).

Example 8.4.4 A digital computer has a memory unit of $64 \text{ K} \times 16$ and a cache memory of 1 K words. The cache uses direct mapping with a block size of four words. How many bits there in the tag index, block and word field of the address format?

Solution: Word bits: Number of word bits = $\log_2 2^2 = 2$ -bits

Block bits: Number of block = Cache size / Words in each block

Number of block bits = $\log_2 256 = \log_2 2^8 = 8$ bits.



Tag bits: Number of bits to address main memory

$$= \log_2 64 \text{ K} = \log_2 2^{16} = 16 \text{ bits}$$

$$\text{Number of Tag bits} = 16 - 8 - 2 = 6 \text{ bits}$$

Example 8.4.5 A two way set associative cache memory uses block of four words. The cache can accommodate a total of 2048 words from main memory. The main memory size is $128 \text{ K} \times 32$.

i) How many bits are there in the tag index, block and word field of address format?

ii) What is size of cache memory?

Solution: Number of bits in main memory address = $\log_2 128K = \log_2 2^{17} = 17$ bit

Number of blocks in the cache memory = $2048/4 = 512$ blocks

Number of sets in the cache memory = $512/2 = 256$ sets

Number of bits in set field = $\log_2 256 = \log_2 2^8 = 8$ bits

Number of bits in word field = $\log_2 4 = \log_2 2^2 = 2$ bits

Number of bits in tag field = $17 - 8 - 2 = 7$ bits



Example 8.4.6 A direct mapped cache has the following parameters: cache size = 1 K words, Block size 128 words and main memory size is 64 K words. Specify the number of bits in TAG, BLOCK and WORD in main memory address.

Solution : Word bits = $\log_2 128 = 7$ bits

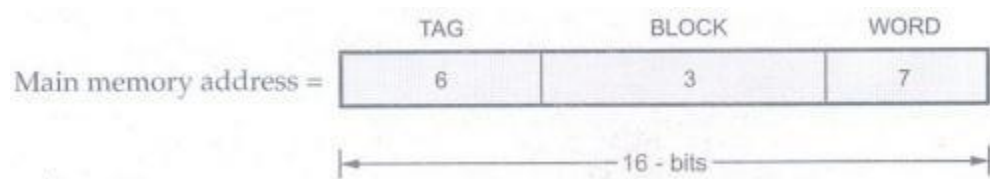
Number of blocks = cache size / words in each block = $1K/128 = 8$

Number of block bits = $\log_2 8 = 3$ -bits

Number of address bits to address main memory

$$= \log_2 64K = \log_2 2^{16} = 16 \text{ bits}$$

$$\text{Tag bits} = 16 - 3 - 7 = 6 \text{ bits}$$



Example 8.4.7 How many total bits are required for a direct-mapped cache with 16 kb of data and 4-word blocks, assuming a 32-bit address?

Dec.-17, May-19, Marks 2

Solution:

$$16 \text{ kb} = 4K \text{ words} = 2^{12} \text{ words}$$

$$\text{Block size of 4 words} = 2^{10} \text{ blocks}$$

Each block has $4 \times 32 = 128$ bits of data + tag + valid bit

$$\text{Tag + valid bit} = (32 - 10 - 2 - 2) + 1 = 19$$

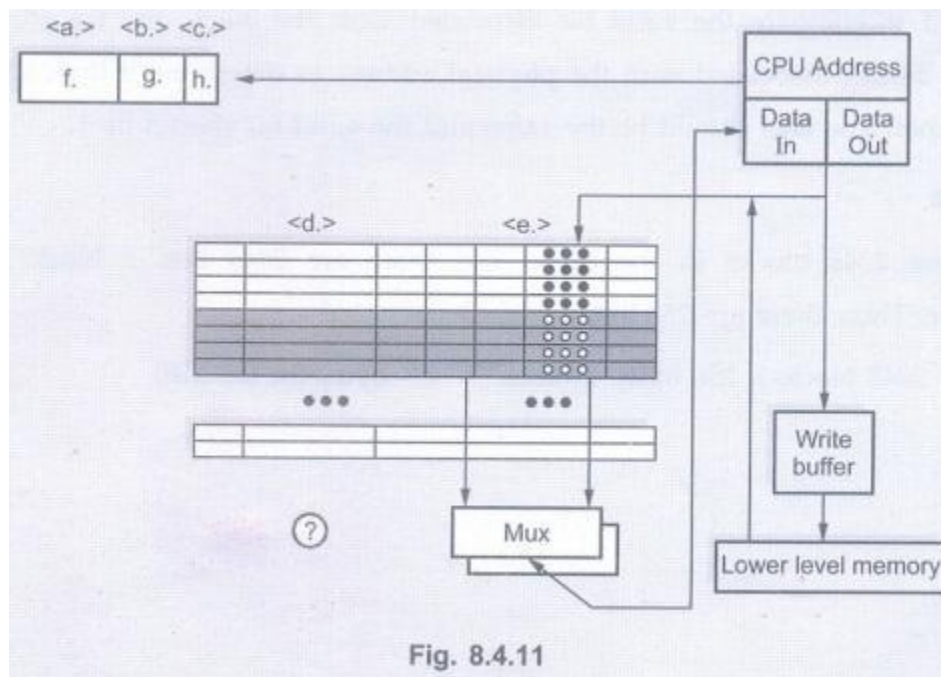
$$\text{Total cache size} = 2^{10} (128 + 19) = 2^{10} \times 147$$

Therefore, 147 kb are needed for the cache.

Example 8.4.8 You have been asked to design a cache with the following properties:

- 1) Data words are 32 bits each.
- 2) A cache block will contain 2048 bits of data.
- 3) The cache is direct mapped.
- 4) The address supplied from the CPU is 32 bits long.
- 5) There are 2048 blocks in the cache.
- 6) Addresses are to the word.

In the below Fig.8.4.11, there are 8 fields (labeled a,b,c,d,e,f,g and h), you will need to indicate the proper name or number of bits for a particular portion of this cache configuration. **AU May-19, Marks 8**



Solution:

f. (name) - You are being asked to show what part of a physical address form the index, offset, and tag. $\langle f \rangle$ refers to the most significant bits of the address - so this is the tag.

g. (name) - It follows that the next part of the address is the index.

h. (name) - The least significant bits form the offset.

c. (number) - There are 2^{11} bits / block and there are 2^5 bits / word. Thus there are 2^6 words / block so we need 6 bits of offset.

b. (number) - There are 2^{11} blocks and the cache is direct mapped (or "1-way set associative"). Therefore, we need 11 bits of index.

a. (number) - The remaining bits form the tag. Thus, $32 - 6 - 11 = 15$ bits of tag.

d. (number) - Field $\langle d \rangle$ refers to the fact that a tag must be stored in each block. Thus, 15 bits are kept in each block.

e. (number) Field $\langle e \rangle$ asks you to specify the total number of bits / block. This is 2048.

We need to compare the valid bit associated with the block, the tag stored in the block, and the tag associated with the physical address to determine if the cache entry is useable or not. The tags should be the same and the valid bit should be 1.

Cache Size

There are 2048 blocks in the cache and there are 2048 bits / block. There are 8 bits/byte. Thus, there are 256 bytes / block

$2048 \text{ blocks} \times 256 \text{ bytes / block} = 2^{19} \text{ bytes (or 0.5 MB)}$

Comparison between Mapping Techniques

Sr. No.	Direct-mapping	Associative-mapping	Set-associative-mapping
1.	Each block from the main memory has only one possible location in the cache.	A block of data from main memory can be placed into any cache block position.	A block of data from main memory can go into a particular block location of any direct-mapped cache.
2.	Needs only one comparison.	Needs comparison with all tag bits.	Needs number of comparisons equal to number of blocks per set.
3.	Cache hit ratio decreases if processor needs to access same memory location from two different pages of the main memory frequently.	Cache hit ratio has no effect if processor needs to access same memory location from two different pages of the main memory frequently.	The effect of reduction in cache hit ratio in case of frequent access to the two different pages of the main memory is reduced.
4.	Main memory address is divided into three fields : TAG, BLOCK and WORD.	Main memory address is divided into two fields : TAG and WORD.	Main memory address is divided into three fields : TAG, SET and WORD.
5.	Searching time is less.	Searching time is more.	Searching time increases with number of blocks per set.

Table 8.4.1 Comparison between mapping techniques

Replacement Algorithms

- When a new block is brought into the cache, one of the existing blocks must be replaced, by a new block.
- In case of direct-mapping cache, we know that each block from the main memory has only one possible location in the cache, hence there is no choice. The previous data is replaced by the data from the same memory location from new page of the main memory.
- For associative and set-associative techniques, there is a choice of replacing existing block. The choice of replacement of the existing block should be such that the probability of accessing same block must be very less. The replacement algorithms do the task of selecting the existing block which must be replaced. • There are four most common replacement algorithms:
 - Least-Recently-Used (LRU)
 - First-In-First-Out (FIFO)
 - Least-Frequently-Used (LFU)
 - Random
- **Least-Recently-Used:** In this technique, the block in the set which has been in the cache longest with no reference to it, is selected for the replacement. Since we assume that more-recently used memory locations are more likely to be referenced again. This technique can be easily implemented in the two-way set-associative cache organization.

- **First-In-First-Out:** This technique uses same concept that stack implementation uses in the microprocessors. In this technique, the block which is first loaded in the cache amongst the present blocks in the cache is selected for the replacement.
- **Least-Frequently-Used:** In this technique, the block in the set which has the fewest references is selected for the replacement.
- **Random:** Here, there are no specific criteria for replacement of any block. The existing blocks are replaced randomly. Simulation studies have proved that random replacement algorithm provides only slightly inferior performance to algorithms just discussed.

Example 8.4.9 Consider web browsing application. Assuming both client and server are involved in the process of web browsing application, where can caches be placed to speed up the process? Design a memory hierarchy for the system. Show the typical size and latency at various levels of the hierarchy. What is the relationship between cache size and its access latency? What are the units-of data transfers between hierarchies? What is the relationship between the data location, data size, and transfer latency? **AU: Dec.-18, Marks 7**

Solution: a) Assuming both client and server are involved in the process of web browsing application, caches can be placed on both sides - web browser and server.

b) Memory hierarchy for the system is as follows:

1. Browser cache, size = fraction of client computer disk, latency = local disk = latency.
2. Proxy cache, size=proxy disk, latency = LAN+ proxy disk latencies

3. Server-side cache, size = fraction of server disk, latency = WAN + server disk
 4. Server storage, size = server storage, latency = WAN + server storage. Latency is not directly related to cache size.
- c) The units of data transfers between hierarchies are pages.
- d) Latency grows with page size as well as distance.

Page Replacement Algorithms

- If the required page is not found in the main memory, the page fault occurs and the required page is loaded into the main memory from the secondary memory.
- However, if there is no vacant space in the main memory to copy the required page, it is necessary to replace the required page with one of the existing page in the main memory which is currently not in use.
- Thus we can say that the page replacement is a basic requirement of demand paging.
- There are many different page-replacement algorithms used by various operating systems. These are discussed in the following sections.

FIFO Algorithm

- It is the simplest page-replacement algorithm. A FIFO (First In First Out) replacement algorithm replaces the new page with the oldest page in the main memory.

- For our example reference string, our three frames are initially empty. The first three references (6, 0, 1) cause page faults, and the required pages are brought into these empty frames.

The next reference (2) replaces page 6, because page 6 was brought in first. Since 0 is the next reference and 0 is already in memory, we have no fault for this reference. The first reference to 3 results in page 0 being replaced, since it was the first of the three pages in memory (0, 1 and 2) to be brought in. This process continues as shown in Fig. 8.5.9.

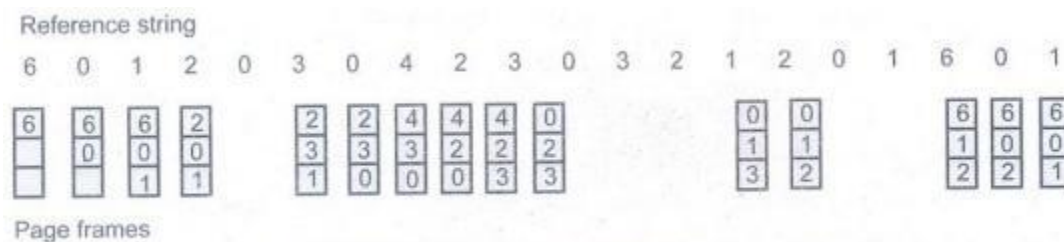


Fig. 8.5.9 FIFO page-replacement algorithm

- The FIFO page-replacement algorithm is easy to understand and program. However, its performance is not always good. It may replace the most needed page as it is oldest page.

Optimal algorithm

- **Advantage:** An optimal page-replacement algorithm has the lowest page-fault rate of all algorithms. It has been called OPT or MIN.
- It simply states that "Replace the page that will not be used for the longest period of time".

- For example, on our sample reference string, the optimal page replacement algorithm would yield nine page faults, as shown in Fig. 8.5.10. The first three references cause faults that fill the three empty frames.

Reference string

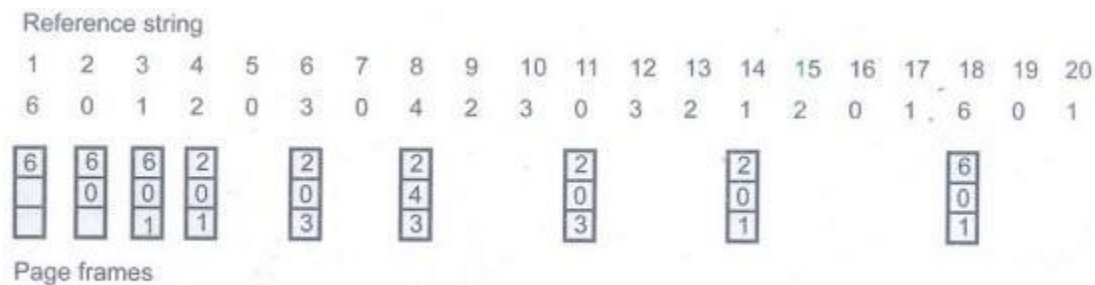


Fig. 8.5.10 Optimal page replacement algorithm

- The reference to page 2 replaces page 6, because 6 will not be used until reference 18, whereas page 0 will be used at 5, and page 1 at 14. The reference to page 3 replaces page 1, as page 1 will be the last of the three pages in memory to be referenced again. With only nine page faults, optimal replacement is much better than a FIFO algorithm, which had 15 faults. (If we ignore the first three, which all algorithms must suffer, then optimal replacement is twice as good as FIFO replacement.) In fact, no replacement algorithm can process this reference string in three frames with less than nine faults.

- **Disadvantage:** Unfortunately, the optimal page replacement algorithm is difficult to implement, because it requires future knowledge of the reference string.

LRU algorithm

- The algorithm which replaces the page that has not been used for the longest period of time is referred to as the Least Recently Used (LRU) algorithm.

- The result of applying LRU replacement to our example reference string is shown in Fig. 8.5.11.

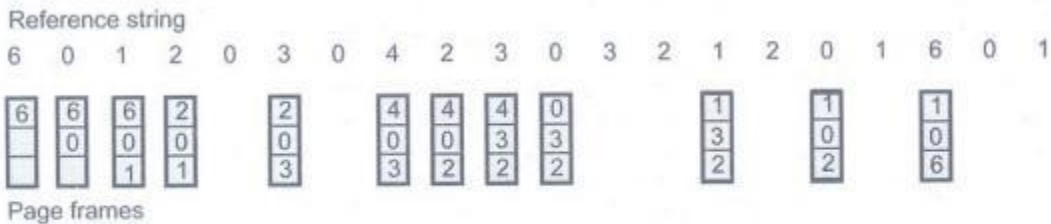


Fig. 8.5.11 LRU page replacement algorithm

- When the reference to page 4 occurs, LRU replacement sees that, of the three frames in memory, page 2 was used least recently. The most recently used page is page 0, and just before that page 3 was used. Thus, the LRU algorithm replaces page 2, not knowing that page 2.
- When it then faults for page 2, the LRU algorithm replaces page 3 since, of the three pages in memory {0, 3, 4}, page 3 is the least recently used.
- LRU replacement with 12 faults is still much better than FIFO replacement with 15.

Counting algorithms

- In the counting algorithm the a counter of the number of references that have been made to each pages are kept, and based on these counts the following two schemes work.
- **LFU algorithm:** The Least Frequently Used (LFU) page replacement algorithm requires that the page with the smallest count be replaced.

- The reason for this selection is that an actively used page should have a large reference count.
- This algorithm suffers from the situation in which a page is used heavily during the initial phase of a process, but then is never used again.
- **MFU algorithm:** The Most Frequently Used (MFU) page replacement algorithm is based on the argument that the page with the smallest count was probably just brought in and has yet to be used.

Example 8.5.5: Explain page replacement algorithms. Find out page fault for following string using LRU method

60 12 0 30 4 2 30 321 20 15

Consider page frame size 3.

Solution:

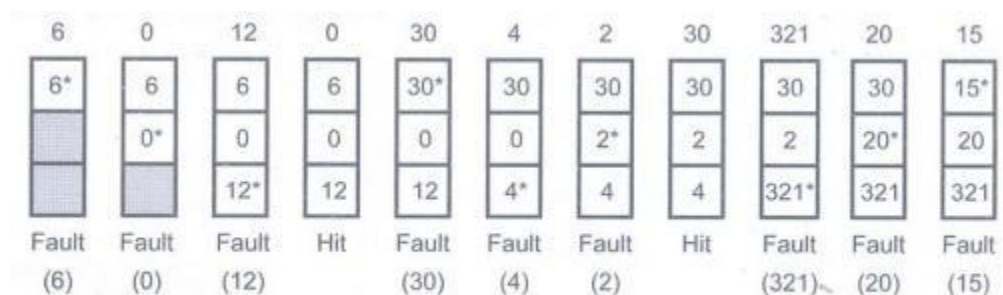


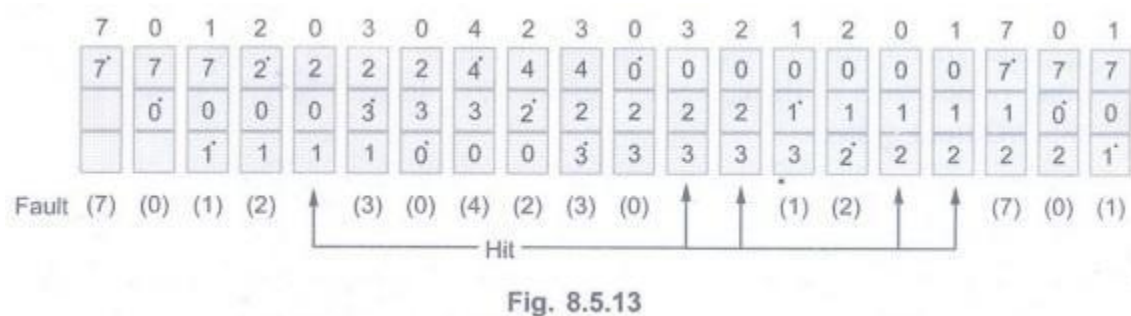
Fig. 8.5.12

Total page faults= 09

Example 8.5.6: Explain page replacement algorithm. Find out page fault for following string using LRU method. Consider page frame size 3.

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Solution :Reference string



Total page faults = 15

Virtual Memory Concept

- The virtual memory technique is used to extend the apparent size of the physical memory.
- It uses secondary storage such as disks, to extend the apparent size of the physical memory.
- Virtual memory is a concept that allows user to construct programs as large as auxiliary memory.
- When a program does not completely fit into the main memory, it is divided into segments. The segments which are currently being executed are kept in the main memory and remaining segments are stored in the secondary storage devices, such as a magnetic disk.

- If an executing program needs a segment which is not currently in the main memory, the required segment is copied from the secondary storage device.
- When a new segment of a program is to be copied into a main memory, it must replace another segment already in the memory.
- In virtual memory, the addresses that processor issues to access either instruction or data are called virtual or logical address. The set of such addresses is called address space. These addresses are translated into physical addresses by a combination of hardware and software components.
- The set of physical addresses or locations is called the memory space.

Concept of Paging

- Fig. 8.5.1 shows a typical memory organization that implements virtual memory. The memory management unit controls this virtual memory system. It translates virtual address into physical addresses.
- A simple method for translating virtual addresses into physical addresses is to assume that all programs and data are composed of fixed length unit called pages, as shown in the Fig. 8.5.2.

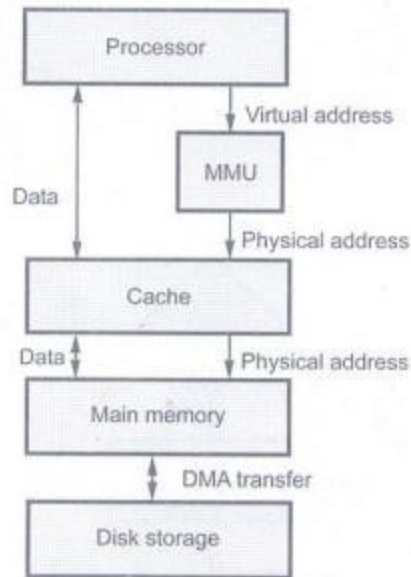


Fig. 8.5.1 Virtual memory organization

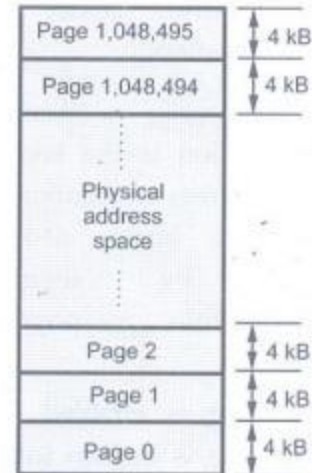


Fig. 8.5.2 Paged organization of the physical address space

- Pages constitute the basic unit of information that is moved between the main memory and the disk whenever the page translation mechanism determines that a swapping is required.

Virtual to Physical Address Translation

- Involves two phases: Segment translation and page translation.

Segment translation :

A logical address (also known as virtual address) consists of a selector and an offset. A selector is the contents of a segment register.

- Every segment selector has a linear base address associated with it, and it is stored in the segment descriptor. A selector is used to point a descriptor for the segment in a table of descriptors. The linear base address from the descriptor is then added to the offset to generate the linear address. This process is known as

segmentation or segment translation. If paging unit is not enabled then the linear address corresponds to the physical address. But if paging unit is enabled, paging mechanism translates the linear address space into the physical address space by paging translation.

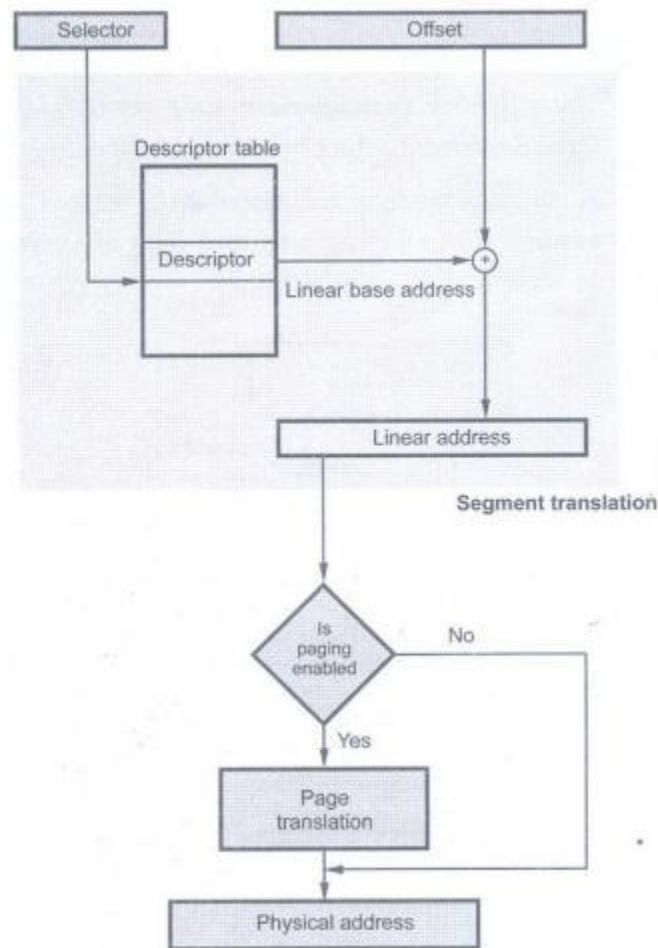


Fig. 8.5.3 Address translation

Page translation

- Page translation is the second phase of address translation. It transforms a linear address generated by segment translation into a physical address.

- When paging is enabled, the linear address is broken into a virtual page number and a page offset.

Fig. 8.5.4 shows the translation of the virtual page number to a physical page number. The physical page number constitutes the upper portion of the physical address, while the page offset, which is not changed, constitutes the lower portion. The number of bits in the page offset field decides the page size.

- The page table is used to keep the information about the main memory location of each page. This information includes the main memory address where the page is stored and the current status of the page.
- To obtain the address of the corresponding entry in the page table the virtual page number is added with the contents of page table base register, in which the starting address of the page table is stored. The entry in the page table gives the physical page number, in which offset is added to get the physical address of the main memory.

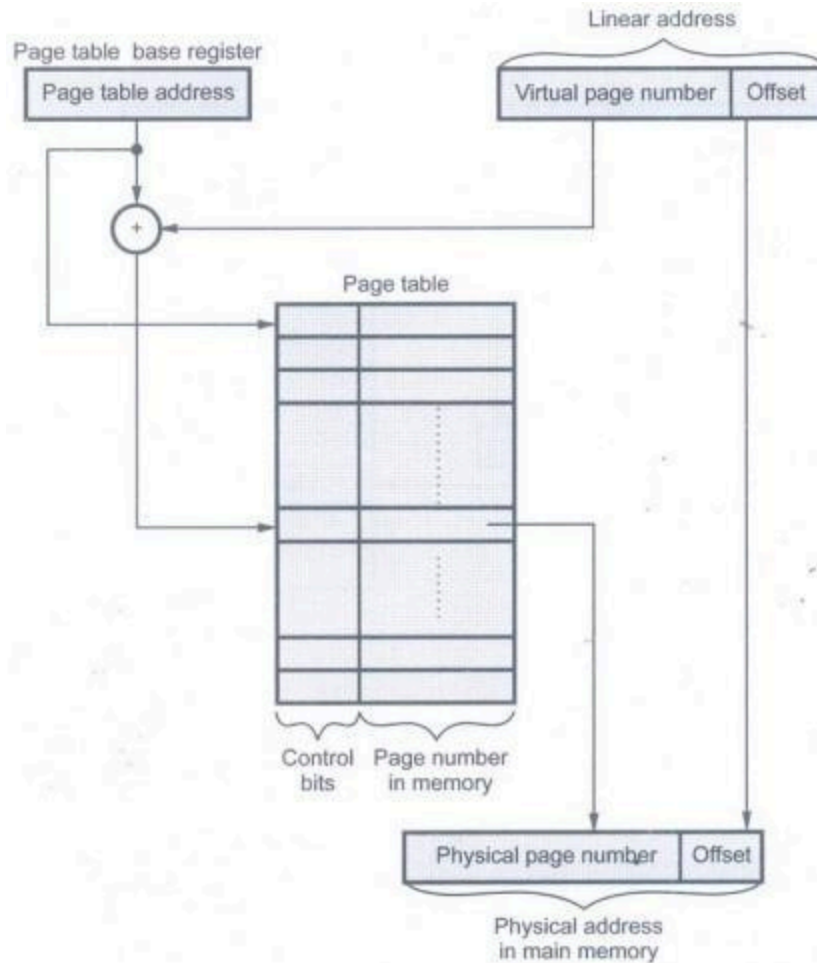


Fig. 8.5.4 Linear to physical address translation

Example 8.5.1 The logical address space in a computer system consists of 128 segments. Each segment can have up to 32 pages of 4 K words each. Physical memory consists of 4 K blocks of 4 K words in each. Formulate the logical and physical address formats.

Solution :

Number of bits for segment address = $\log_2 128 = \log_2 2^7 = 7$ bits

Number of bits for page address = $\log_2 32 = \log_2 2^5 = 5$ bits

Number of bits for word address = $\log_2 4096 = 2^{12} = 12$ bits

Number of bits for block address = $\log_2 4096 = \log_2 2^{12} = 12$ bits

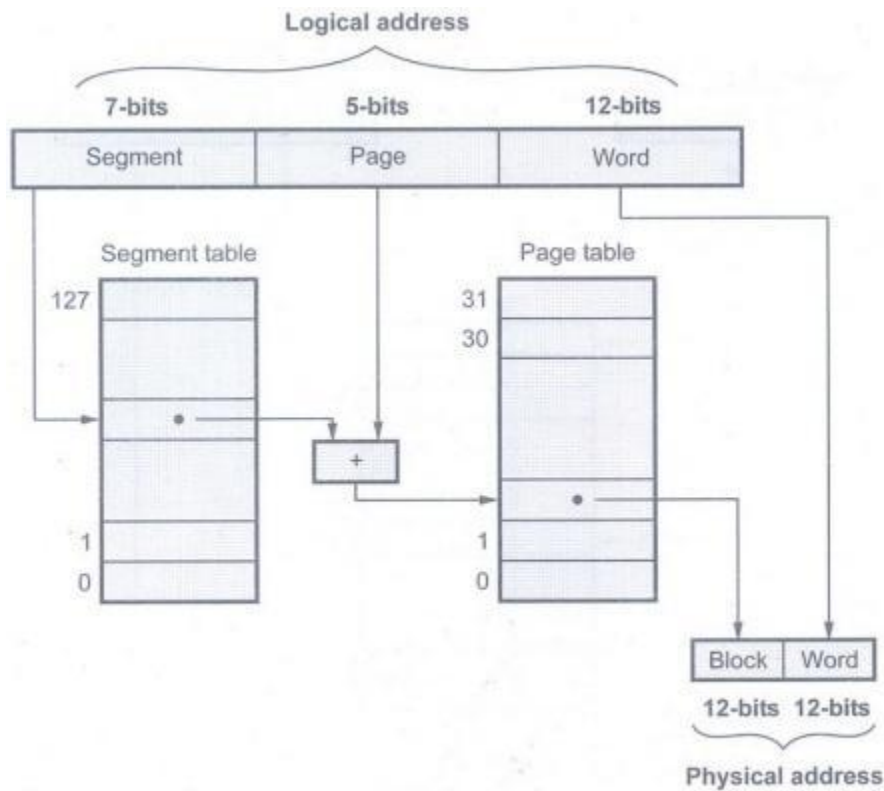


Fig. 8.5.5

Example 8.5.2 An address space is specified by 32 bits and corresponding memory space by 24 bits.

- How many words are there in the address space?
- How many words are there in the memory space?
- If a page consists of 4 K words, how many pages and blocks are there in the systems.

Solution :

i) Words in the address space $2^{32} = 4 \text{ G words}$

ii) Words in the memory space $= 2^{24} = 16 \text{ M words}$

iii) Number of pages $= \text{Words in address space} / \text{Words per page} = 4 \text{ G words} / 4 \text{ K words} = 1 \text{ M pages}$

iv) Number of blocks $= \text{Words in memory space} / \text{Words per page/block} = 16 \text{ M words} / 4 \text{ K words} = 4 \text{ K words}$

Example 8.5.3 An address space is specified by 24 bits and the corresponding memory space by 16 bits. How many words are there in the virtual memory and in the main memory ? **AU May-13, Marks 2**

Solution: Words in the address space, i.e., in the virtual memory

$= 2^{24} = 16 \text{ M words}$

Words in the memory space, i.e., in the main memory

$= 2^{16} = 64 \text{ K words}$

Translation Lookaside Buffer (TLB)

- To support demand paging and virtual memory processor has to access page table which is kept in the main memory. To reduce the access time and degradation of performance, a small portion of the page table is accommodated in the memory management unit. This portion is called Translation Lookaside Buffer (TLB).

- It is used to hold the page table entries that corresponds to the most recently accessed pages. When processor finds the page table entries in the TLB it does not have to access page table and saves substantial access time.
- The Fig. 8.5.6 shows a organization of a TLB where the associative mapping technique is used.

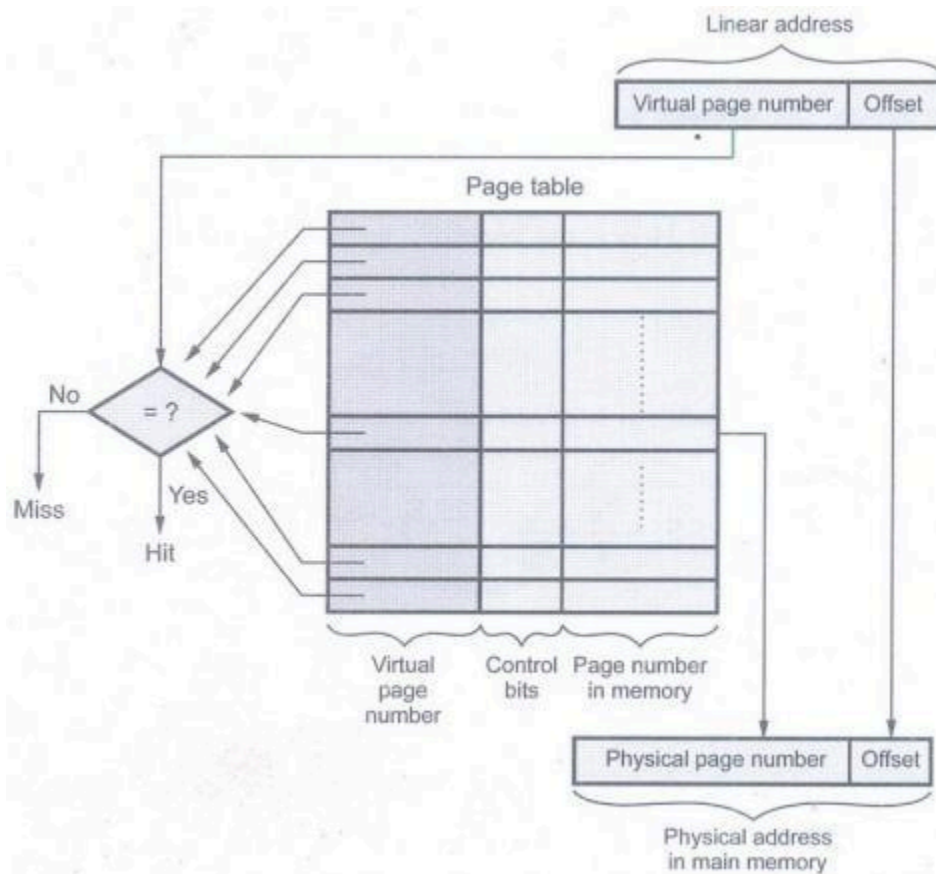


Fig. 8.5.6 TLB operation

- As shown in the Fig. 8.5.6, a given virtual address is compared with the TLB entries for the reference page by MMU. If the page table entry for this page is found in the TLB, the physical address is obtained immediately.

- If entry is not found, there is a miss in the TLB, then the required entry is obtained from the page table in the main memory and the TLB is updated.
- It is important that the contents of the TLB be coherent with the contents of page tables in the memory. When page table entries are changed by an operating system, it must invalidate the corresponding entries in the TLB. One of the control bits in the TLB is provided for this purpose. When an entry is invalidated, the TLB acquires the new information as a part of the MMU's normal response to access misses.

Page Fault and Demand Paging

- If the page required by the processor is not in the main memory, the page fault occurs and the required page is loaded into the main memory from the secondary storage memory by special routine called page fault routine.
- This technique of getting the desired page in the main memory is called demand paging.
- A demand-paging system is similar to a paging system with swapping. A swapper never swaps a page into memory unless that page is needed. Since we are now viewing a program as a sequence of pages, rather than one large contiguous address space, the use of the term swap is technically incorrect. A swapper manipulates entire programs, whereas a pager is concerned with the individual pages of a program. Hence term pager is usually used, rather than swapper, in connections with demand paging.
- When a program is to be swapped in, the pager guesses which pages will be used before the program is swapped out again. Instead of swapping in a whole process,

the pager brings only those necessary pages into memory. Thus, it avoids reading into memory pages that will not be used any way, decreasing the swap time and the amount of physical memory needed.

- With this scheme, we need some form of hardware support to distinguish between those pages that are in memory and those pages that are on the disk. Fig. 8.5.8 shows hardware to implement demand paging.

- As shown in the Fig. 8.5.8, the valid-invalid bits are used to distinguish between those pages that are in memory and those pages that are on the disk. When bit is set to "valid", it indicates that the associated page is both legal and in memory. If the bit is set to "invalid", it indicates that the page either is not valid (that is, not in the logical address space of the process), or is valid but is currently on the disk. If the process tries to use a page that was not brought into memory, access to a page marked invalid causes a page-fault trap. This trap is the result of the operating system's failure to bring the desired page into memory. The procedure for handling this page fault is illustrated in Fig. 8.5.8.

- This procedure goes through following steps:

1. It checks an internal table for this program, to determine whether the reference was a valid or invalid memory access.
2. If the reference is invalid, it terminates the process. If it is valid, but the referenced page is not available in the physical memory, trap is activated.
3. It finds a free frame.

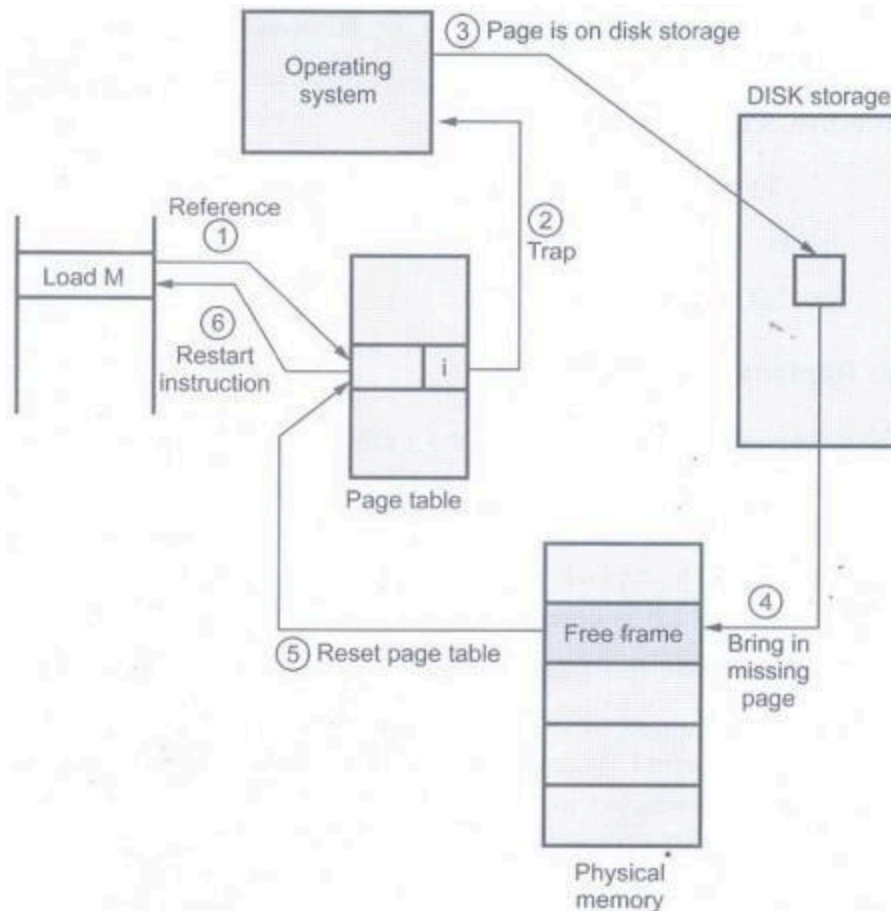


Fig. 8.5.8 Steps in handling a page fault

Steps in handling Page fault

4. It schedules a disk operation to read the desired page into the newly allocated frame.
5. When the disk read is complete, it modifies the internal table kept with the program and the page table to indicate that the page is now in memory.
6. It restarts the instruction that was interrupted by the illegal address trap. The program can now access the page as though it had always been in memory.

- The hardware to implement demand paging is the same as the hardware for paging and swapping :

Page table: This table has the ability to mark an entry invalid through a valid-invalid bit or special value of protection bits.

Secondary memory: This memory holds those pages that are not present in main memory. The secondary memory is usually a hard-disk. It is known as the swap device, and the section of disk used for this purpose is known as swap space or backing store.

Example 8.5.4: Calculate the effective address time if average page-fault service time of 20 milliseconds and a memory access time of 80 nanoseconds. Let us assume the probability of a page fault 10 %. **Dec.-09, Marks 2**

Solution: Effective access time is given as

$$=(1 - 0.1) \times (80) + 0.1 (20 \text{ milliseconds})$$

$$=(1 - 0.1) \times 80 + 0.1 \times 20,000,000 = 72 + 2,000,000 \text{ (nanoseconds)}$$

$$=2,000,072 \text{ (nanoseconds)}$$

Direct Memory Access (DMA)

AU: Dec.-06,07,08,11,15,16,18, May-07,09,12,14,16,17,19

- In software control data transfer, processor executes a series of instructions to carry out data transfer. For each instruction execution fetch, decode and execute phases are required. Fig. 8.10.1 gives the flowchart to transfer data from memory to I/O device.

- Thus to carry out these tasks processor requires considerable time. So this method of data transfer is not suitable for large data transfers such as data transfer from magnetic disk or optical disk to memory.

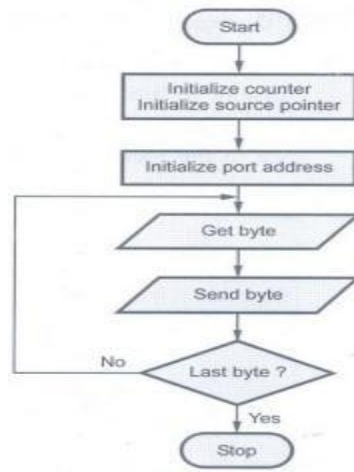


Fig. 8.10.1

Flowchart to transfer data from memory to I/O device.

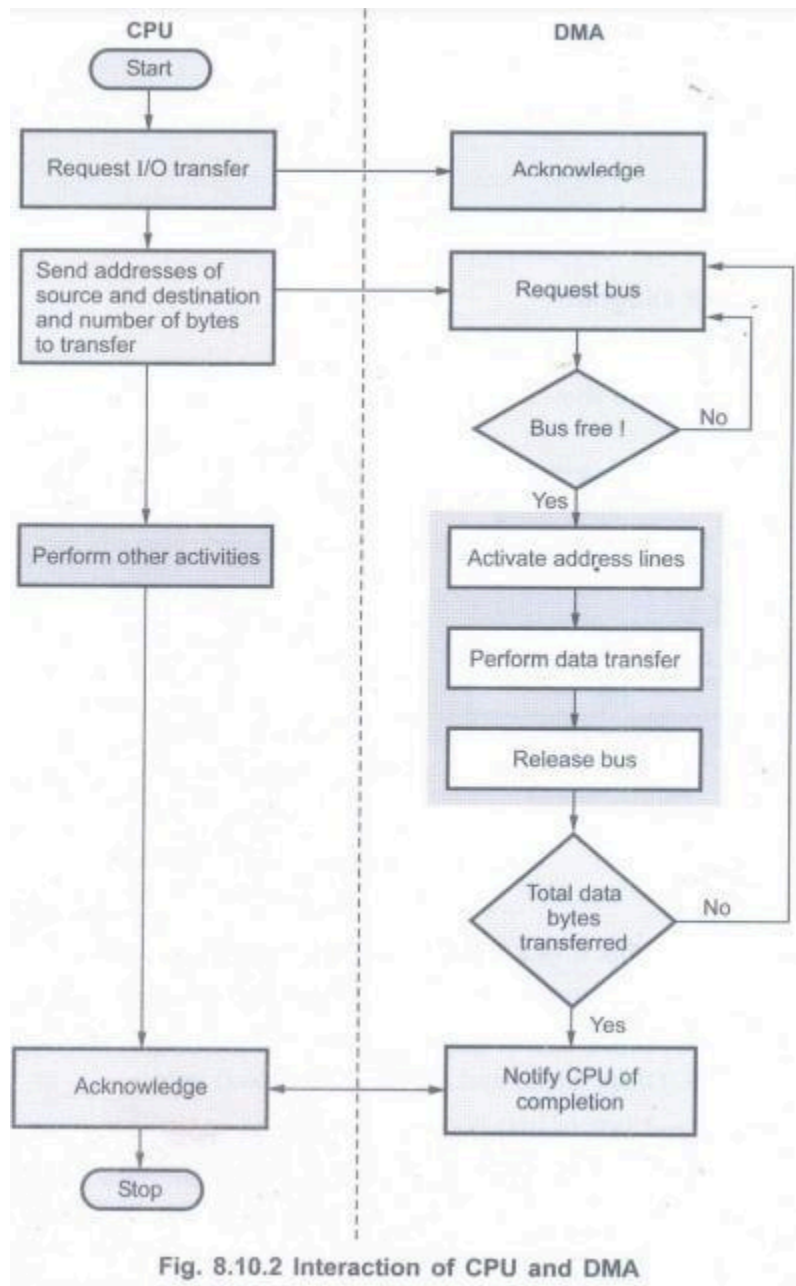
Drawbacks in programmed I/O and interrupt driven I/O

1. The I/O transfer rate is limited by the speed with which the CPU can test and service a device.
 2. The time that the CPU spends testing I/O device status and executing a number of instructions for I/O data transfers can often be better spent on other processing tasks.
- To overcome above drawbacks an alternative technique, hardware controlled data transfer can be used.

DMA Operation

- DMA controlled data transfer is used for large data transfer. For example to read bulk amount data from disk to main memory.
- To read a block of data from the disk processor sends a series of commands to the disk controller device telling it to search and read the desired block of data from the disk.
- When disk controller is ready to transfer first byte of data from disk, it sends DMA request DRQ signal to the DMA controller.
- Then DMA controller sends a hold request HRQ, signal to the processor HOLD input. The processor responds this HOLD signal by floating its buses and sending out a hold acknowledge signal HLDA, to the DMA controller.
- When the DMA controller receives the HLDA signal, it takes the control of system bus.
- When DMA controller gets control of the buses, it sends the memory address where the first byte of data from the disk is to be written. It also sends a DMA acknowledge, DACK signal to the disk controller device telling it to get ready to output the byte.
- Finally, it asserts both the I/O read and memory write signals on the control bus. Asserting the I/O read signal enables the disk controller to output the byte of data from the disk on the data bus and asserting the memory write signal enables the addressed memory to accept data from the data bus. In this technique data is transferred directly from the disk controller to the memory location without passing through the processor or the DMA controller.
- Thus, the CPU is involved only at the beginning and end of the transfer.

- After completion of data transfer, the HOLD signal is deasserted to give control of all buses back to the processor.
- The Fig. 8.10.2 shows the interaction between processor and DMA discussed above.

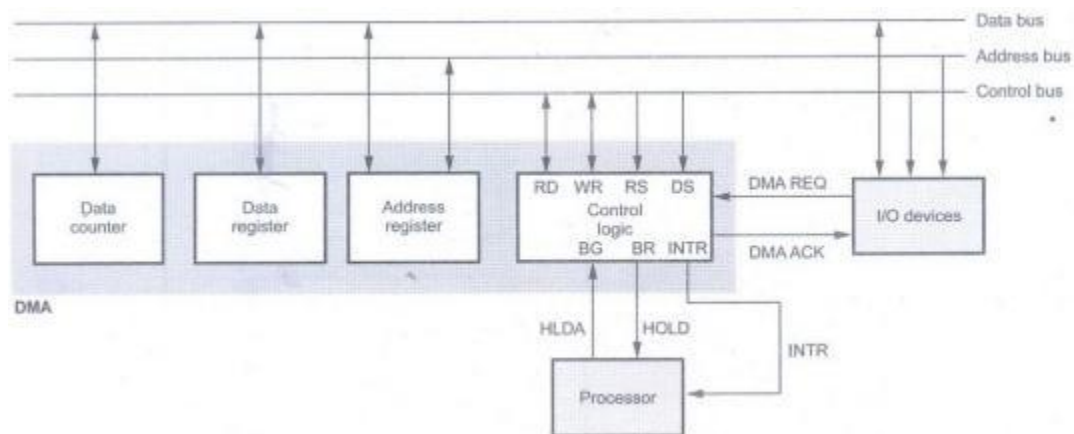


Comparison of I/O Program Controlled Transfer and DMA Transfer

Sr. No.	I/O Program controlled transfer	DMA transfer
1.	It is software controlled data transfer	Hardware controlled data transfer.
2.	Data transfer speed is low.	Data transfer speed is high.
3.	CPU is involved in the transfer.	CPU is not involved in the transfer.
4.	Extra hardware is not required.	DMA controller is required to data transfer.
5.	During data transfer data is routed through processor	During data transfer data does not routed through processor.

DMA Block Diagram

- For performing the DMA operation, the basic blocks required in a DMA channel/controller are shown in Fig. 8.10.3.



- DMA controller communicates with the CPU via the data bus and control lines.
- The registers in DMA are selected by the CPU through the address bus by enabling the DS (DMA select) and RS (Register select) inputs.

- The RD (Read) and WR (write) inputs are bidirectional.
- When the BG (bus grant) input is 0, the CPU can communicate with the DMA registers through the data bus to read from or write the DMA registers RD and WR signals are input signals for DMA.
- When $BG = 1$, the CPU has relinquished the buses and the DMA can communicate directly with the memory by specifying an address in the address bus and activating the RD or WR signals (RD and WR are now output signals for DMA). DMA consists of data count; data register, address register and control logic.
- Data counter register stores the number which gives the number data transfers to be done in one DMA cycle. It is automatically decremented after each word transfer.
- Data register acts as buffer whereas address register initially holds the starting address of the device. Actually, it stores the address of the next word to be transferred. It is automatically incremented or decremented after each word transfer.
- After each transfer, data counter is tested for zero. When the data count reaches zero, the DMA transfer halts.
- The DMA controller is normally provided with an interrupts capability, in which case it sends an interrupt to processor to signal the end of the I/O data transfer.

Data Transfer Modes

- DMA controller transfers data in one of the following three modes :

- Single transfer mode (cycle stealing)
- Block transfer mode
- Demand or burst transfer mode

Single transfer mode (Cycle stealing mode)

In this mode device can make only one transfer (byte or word). After each transfer DMAC gives the control of all buses to the processor. Due to this processor can have access to the buses on a regular basis.

It allows the DMAC to time share the buses with the processor, hence this mode is most commonly used.

The operation of the DMA in a single transfer mode is as given below:

1. I/O device asserts DRQ line when it is ready to transfer data.
2. The DMAC asserts HLDA line to request use of the buses from the processor.
3. The processor asserts HLDA, granting the control of buses to the DMAC.
4. The DMAC asserts $\overline{DACK}$ to the requesting I/O device and executes DMA bus cycle, resulting data transfer.
5. I/O device deasserts its DRQ after data transfer of one byte or word.
6. DMA deasserts $\overline{DACK}$ line.
7. The word/byte transfer count is decremented and the memory address is incremented.

8. The HOLD line is desasserted to give control of all buses back to the processor.
9. HOLD signal is reasserted to request the use of buses when I/O device is ready to transfer another byte or word. The same process is then repeated until the last transfer.
10. When the transfer count is exhausted, terminal count is generated to indicate the end of the transfer.

Block transfer mode

In this mode device can make number of transfers as programmed in the word count register. After each transfer word count is decremented by 1 and the address is decremented or incremented by 1. The DMA transfer is continued until the word count "rolls over" from zero to FFFFH, a Terminal Count (TC) or an external END of Process ($\overline{\text{EOP}}$) is encountered. Block transfer mode is used when the DMAC needs to transfer a block of data.

The operation of DMA in block transfer mode is as given below:

1. I/O device asserts DRQ line when it is ready to transfer data.
2. The DMAC asserts HLDA line to request use of the buses from the microprocessor.
3. The microprocessor asserts HLDA, granting the control of buses to the DMAC.
4. The DMAC asserts $\overline{\text{DACK}}$ to the requesting I/O device and executes DMA bus cycle, resulting data transfer.
5. I/O device deasserts its DRQ after data transfer of one byte or word.

6. DMA deasserts $\overline{\text{DACK}}$ line.

7. The word/byte transfer count is decremented and the memory address is incremented.

8. When the transfer count is exhausted, the data transfer is not complete and the DMAC waits for another DMA request from the I/O device, indicating that it has another byte or word to transfer. When DMAC receives DMA request steps are repeated.

9. If the transfer count is not exhausted, the data transfer is complete then DMAC deasserts the HOLD to tell the microprocessor that it no longer needs the buses.

10. Microprocessor then deasserts the HLDA signal to tell the DMAC that it has resumed control of the buses.

Demand transfer mode

In this mode the device is programmed to continue making transfers until a TC or external $\overline{\text{EOP}}$ is encountered or until DREQ goes inactive.

The operation of DMA in demand transfer mode is as given below:

1. I/O device asserts DRQ line when it is ready to transfer data.
2. The DMAC asserts HLDA line to request use of the buses from the microprocessor.
3. The microprocessor asserts HLDA, granting the control of buses to the DMAC.

4. The DMAC asserts $\overline{DACK}$ to the requesting I/O device and executes DMA bus cycle, resulting data transfer.
5. I/O device deasserts its DRQ after data transfer of one byte or word.
6. DMA deasserts $\overline{DACK}$ line.
7. The word/byte transfer count is decremented and the memory address is incremented.
8. The DMAC continues to execute transfer cycles until the I/O device deasserts DRQ indicating its inability to continue delivering data. The DMAC deasserts HOLD signal, giving the buses back to microprocessor. It also deasserts $\overline{DACK}$.
9. I/O device can re-initiate demand transfer by reasserting DRQ signal.
10. Transfer continues in this way until the transfer count has been exhausted.

The flowcharts in the Fig. 8.10.4 summarized the three data transfer modes of DMA. (See Fig. 8.10.4 on next page.)

Use of DMA in a Computer System

- The Fig. 8.10.5 (a) shows the use of DMA in a computer system.
- The DMA is used to connect a high-speed network to the computer bus. The DMA control handles the data transfer between high-speed network and the computer system.
- It is also used to transfer data between processor and floppy disk with the help of floppy disk controller.

- Let us see how DMA controller does the data transfer between floppy disk and the processor. The Fig. 8.10.5 (b) shows the interface required for such transfer.

- The sequence of events that takes place during the data transfer are as follows:

- When processor needs some data from the disk, it sends a series of command words to registers inside the floppy disk controller.

- The floppy disk controller then proceeds to find the specified track and sector on the disk.

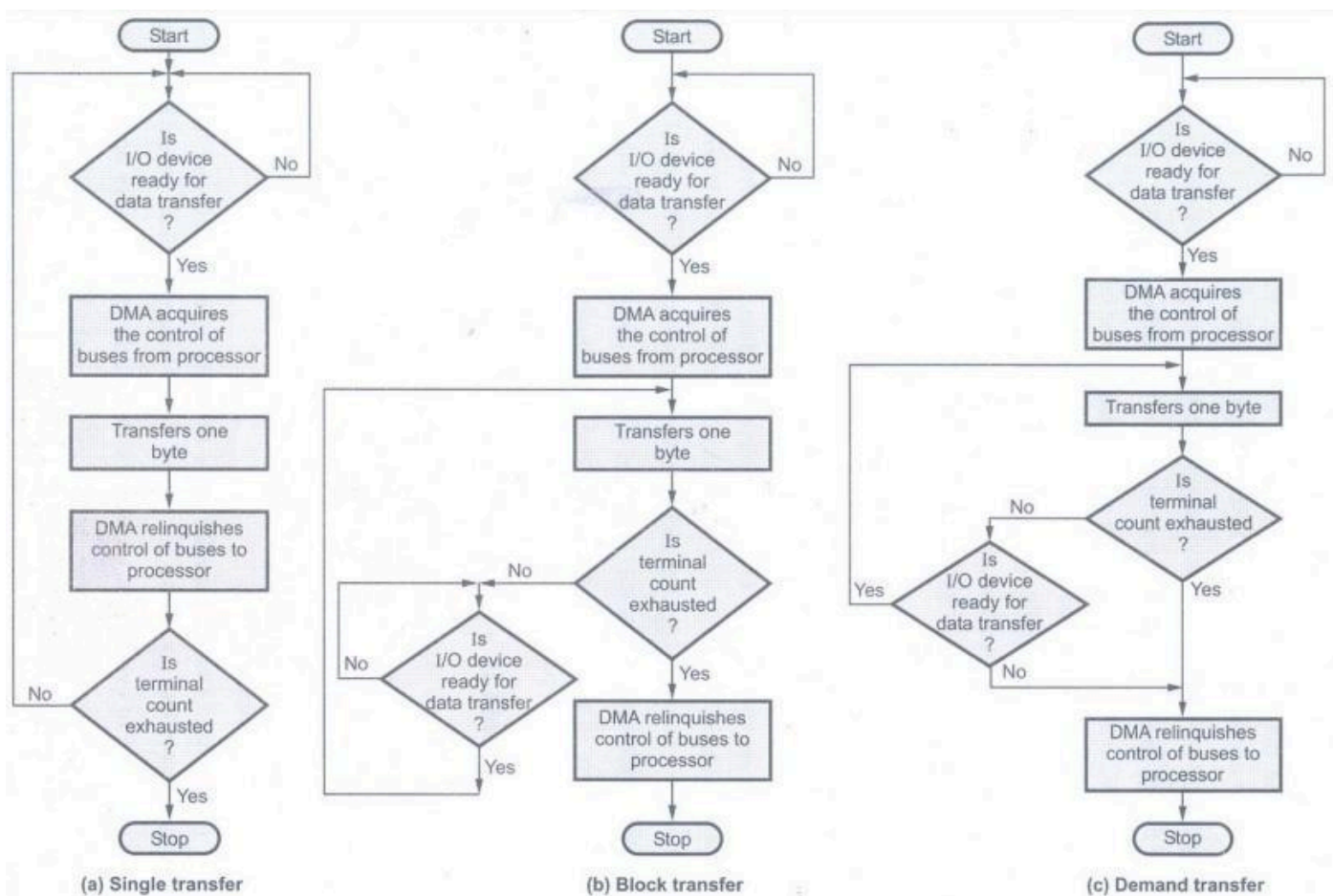


Fig. 8.10.4 Three data transfer modes of DMA

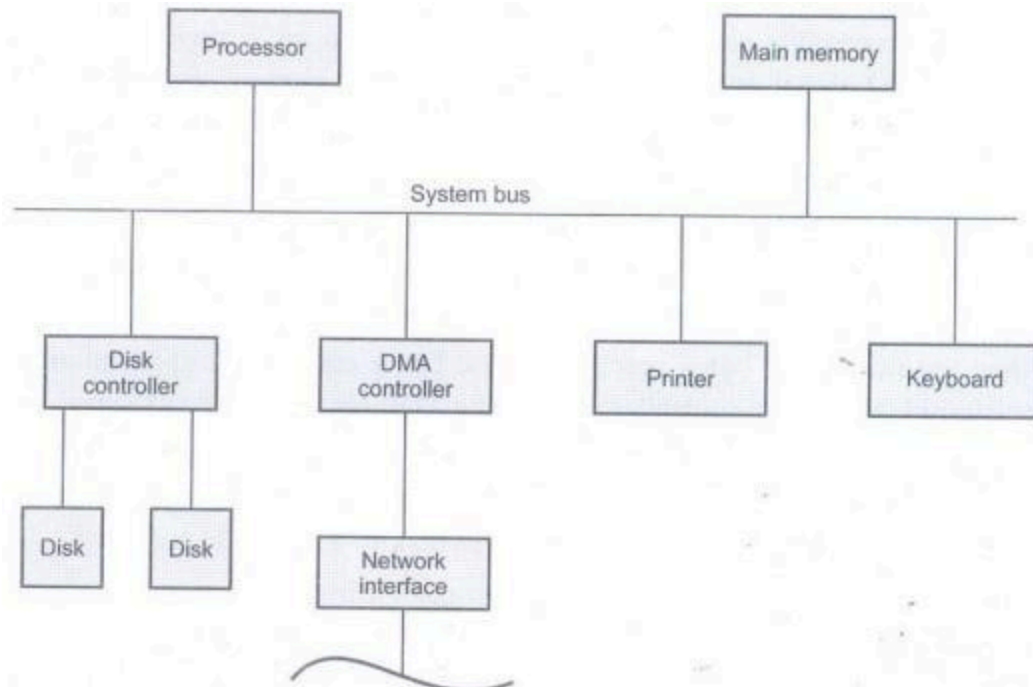


Fig. 8.10.5 (a) Use of DMA controllers in a computer system

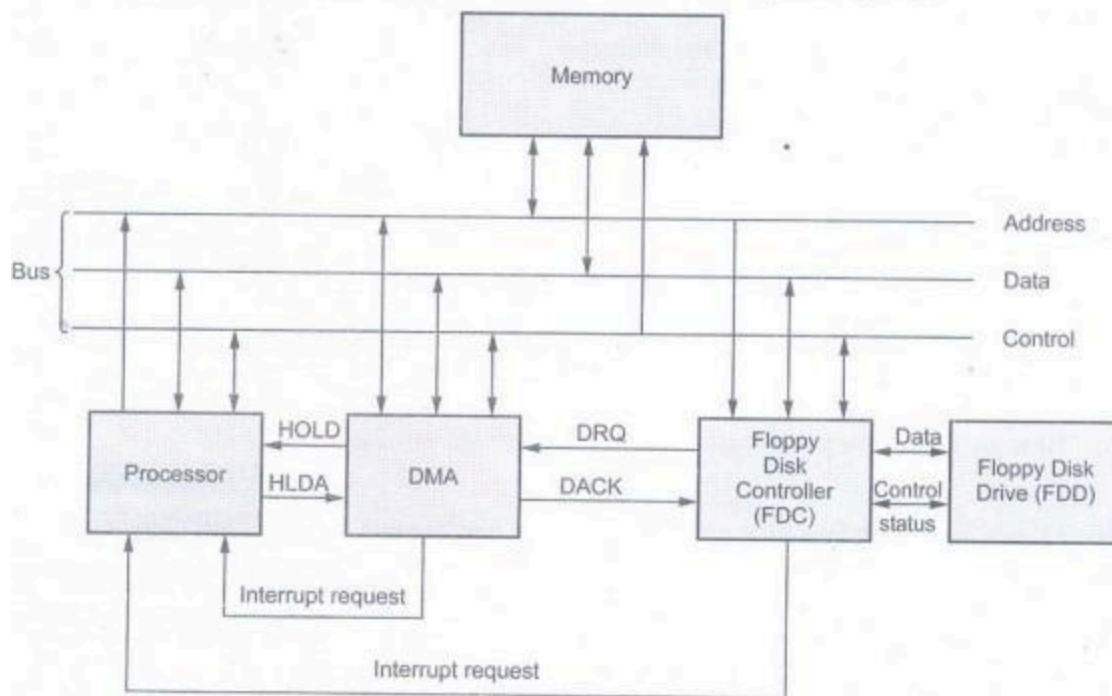


Fig. 8.10.5 (b)

Interface required for such transfer.

- In the mean while processor loads the DMA data counter and address register. The data counter is loaded with count equal to the number of bytes to be transferred to or from the memory. The address register is loaded with the starting address of the memory.
- When the controller reads the first byte of data from a sector, it sends DMA request, DRQ signal to the DMA controller. DMA request sends a hold request signal to the HOLD input of the processor.
- The processor then floats its buses and sends a hold-acknowledge signal to the DMA controller.
- The DMA controller then sends out the first transfer address on the bus and asserts the $\overline{DACK}$ input of the FDC to tell it that the DMA transfer is underway. When the number of bytes specified in the DMA controller initialization has been transferred, the DMA controller asserts the $\overline{EOP}$ (end of process) signal, which is connected to the TC (Terminal Count) input of the FDC.
- This causes FDC to generate interrupt signal to tell the processor that the requested block of data has been read from disk to a buffer in memory.

Similar process is required to write data into the disk.

Accessing I/O

AU May-03, 04, 07, 09, 13, Dec.-04, 06, 07, 10, 14

- The important components of any computer system are CPU, memory and I/O devices (peripherals). The CPU fetches instructions (opcodes and operands/data) from memory, processes them and stores results in memory. The other components of the computer system (I/O devices) may be loosely called the Input/Output system.

- The main function of I/O system is to transfer information between CPU or memory and the outside world.

- The important point to be noted here is, I/O devices (peripherals) cannot be connected directly to the system bus. The reasons are discussed here.

1. A variety of peripherals with different methods of operation are available. So it would be impractical to incorporate the necessary logic within the CPU to control a range of devices.

2. The data transfer rate of peripherals is often much slower than that of the memory or CPU. So it is impractical to use the high speed system bus to communicate directly with the peripherals.

3. Generally, the peripherals used in a computer system have different data formats and word lengths than that of CPU used in it.

- So to overcome all these difficulties, it is necessary to use a module in between system bus and peripherals, called I/O module or I/O system, or I/O interface.

The functions performed by an I/O interface are:

1. Handle data transfer between much slower peripherals and CPU or memory.

2. Handle data transfer between CPU or memory and peripherals having different data formats and word lengths.
3. Match signal levels of different I/O protocols with computer signal levels.
4. Provides necessary driving capabilities - sinking and sourcing currents.

Requirements of I/O System

- The I/O system is nothing but the hardware required to connect an I/O device to the bus. It is also called I/O interface. The major requirements of an I/O interface are :

1. Control and timing
2. Processor communication
3. Device communication
4. Data buffering
5. Error detection

- The important blocks necessary in any I/O interface are shown in Fig. 8.6.1.

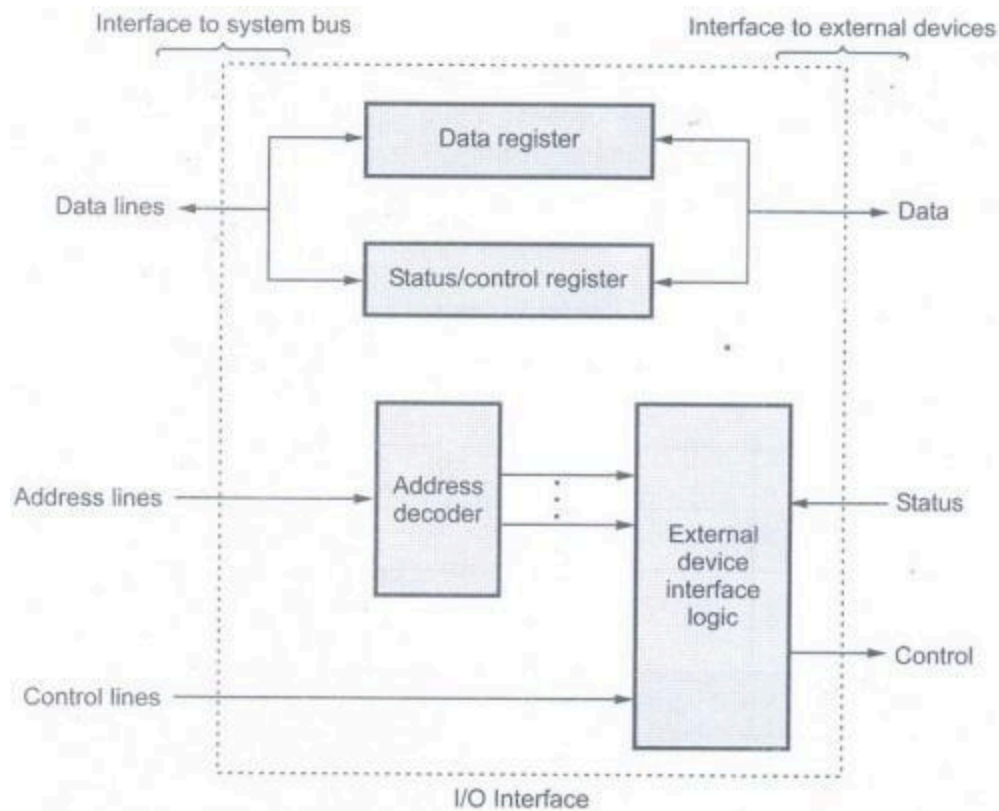
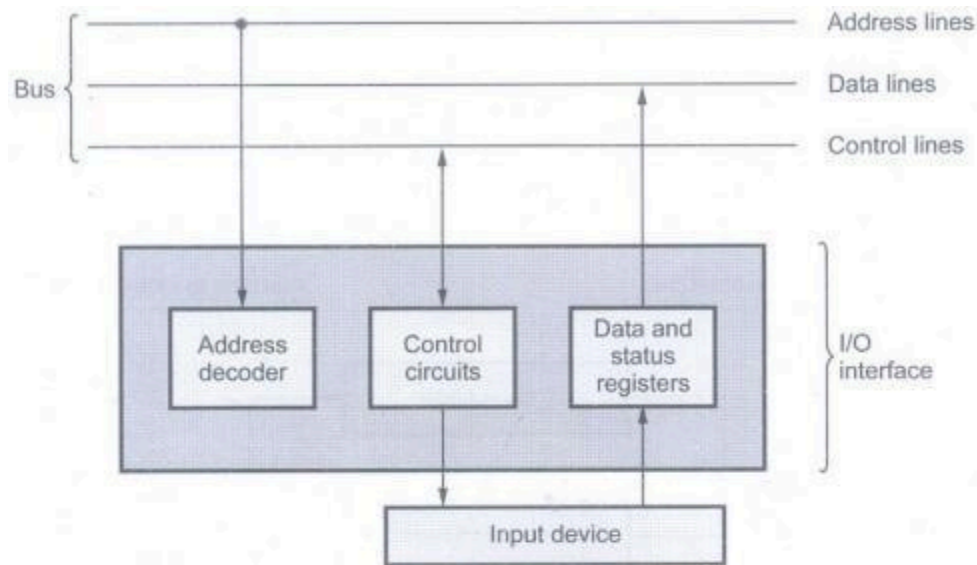


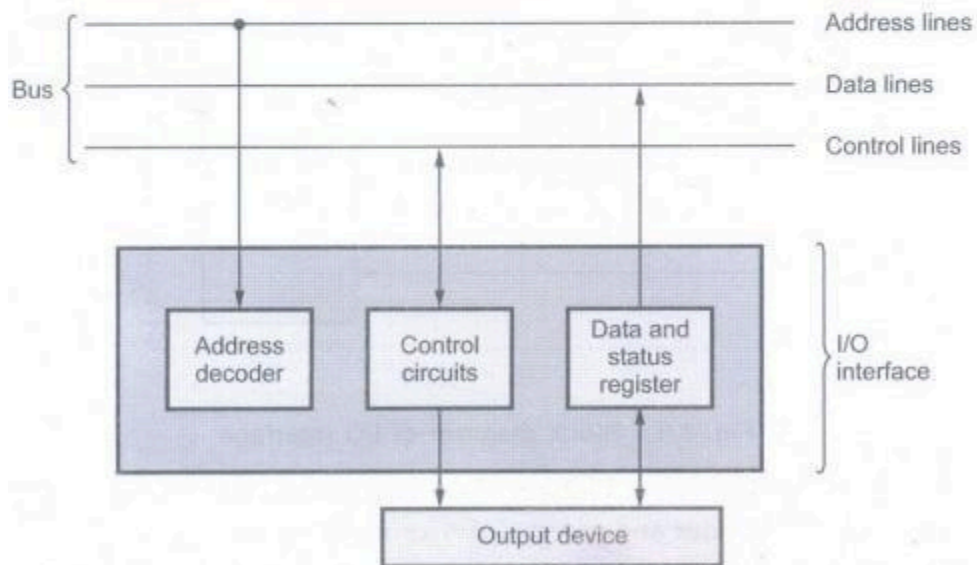
Fig. 8.6.1 Block diagram of I/O interface

- As shown in the Fig. 8.6.1, I/O interface consists of data register, status/control register, address decoder and external device interface logic.
- The data register holds the data being transferred to or from the processor.
- The status/control register contains information relevant to the operation of the I/O device. Both data and status/control registers are connected to the data bus.
- Address lines drive the address decoder. The address decoder enables the device to recognize its address when address appears on the address lines.
- The external device interface logic accepts inputs from address decoder, processor control lines and status signal from the I/O device and generates control

signals to control the direction and speed of data transfer between processor and I/O devices.



(a) I/O interface for input device



(b) I/O interface for output device

Fig. 8.6.2

- The Fig. 8.6.2 shows the I/O interface for input device and output device. Here, for simplicity block schematic of I/O interface is shown instead of detail connections.
- The address decoder enables the device when its address appears on the address lines.
- The data register holds the data being transferred to or from the processor.
- The status register contains information relevant to the operation of the I/O device.
- Both the data and status registers are assigned with unique addresses and they are connected to the data bus.

I/O Interfacing Techniques

I/O devices can be interfaced to a computer system I/O in two ways, which are called interfacing techniques,

- Memory mapped I/O
- I/O mapped I/O

Memory mapped I/O

- In this technique, the total memory address space is partitioned and part of this space is devoted to I/O addressing as shown in Fig. 8.6.3.

- When this technique is used, a memory reference instruction that causes data to be fetched from or stored at address specified, automatically becomes an I/O instruction if that address is made the address of an I/O port.

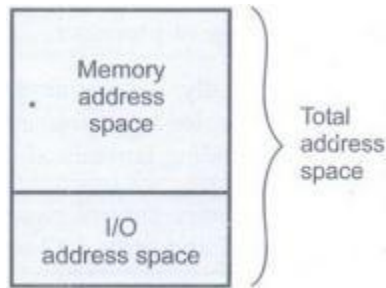


Fig. 8.6.3 Address space

Advantage

- The usual memory related instructions are used for I/O related operations. The special I/O instructions are not required.

Disadvantage

- The memory address space is reduced.

I/O mapped I/O

- If we do not want to reduce the memory address space, we allot a different I/O address space, apart from total memory space which is called I/O mapped I/O technique as shown in Fig. 8.6.4.

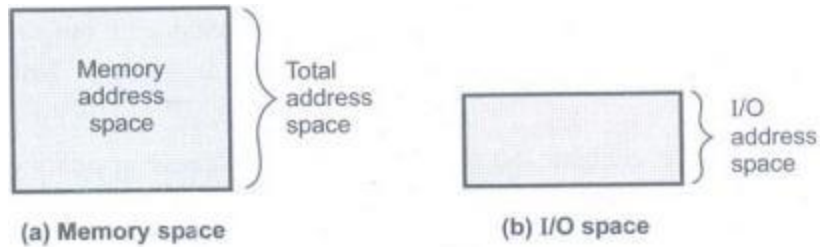


Fig. 8.6.4 Address space

Advantage

- The advantage is that the full memory address space is available.

Disadvantage

- The memory related instructions do not work. Therefore, processor can only use this mode if it has special instructions for I/O related operations such as I/O read, I/O write.

Memory Mapped I/O, I/O Mapped I/O Comparison

Sr. No.	Memory mapped I/O	I/O mapped I/O
1.	Memory and I/O share the entire address range of processor.	Processor provides separate address range for memory and I/O devices.
2.	Usually, processor provides more address lines for accessing memory. Therefore more decoding is required control signals.	Usually, processor provides less address lines for accessing I/O. Therefore, less decoding is required.
3.	Memory control signals are used to control read and write I/O operations.	I/O control signals are used to control read and write I/O operations.

Sr. No.	Memory bus	I/O bus
1.	Memory address bus shares entire address range.	I/O bus shares only I/O address range.
2.	Memory address bus width is greater than I/O address bus width	I/O address bus width is smaller than memory address bus width.
3.	Memory bus includes data bus, address bus and control signals to access memory.	I/O bus includes data bus, address bus and control signals to access I/O.

Types of Data Transfer Techniques

- In I/O data transfer, the system requires the transfer of data between external circuitry and the processor. Different ways of I/O data transfer are:

1. Program controlled I/O or polling control.
2. Interrupt program controlled I/O or interrupt driven I/O.
3. Hardware controlled I/O.
4. I/O controlled by handshake signals.

Program controlled I/O or polling control

- In program controlled I/O, the transfer of data is completely under the control of the processor program. This means that the data transfer takes place only when an

I/O transfer instructions executed. In most of the cases it is necessary to check whether the device is ready for data transfer or not. To check this, processor polls the status bit associated with the I/O device.

Interrupt program controlled I/O or interrupt driven I/O

- In interrupt program controlled approach, when a peripheral is ready to transfer data, it sends an interrupt signal to the processor. This indicates that the I/O data transfer is initiated by the external I/O device.
- When interrupted, the processor stops the execution of the program and transfers the program control to an interrupt service routine.
- This interrupt service routine performs the data transfer.
- After the data transfer, it returns control to the main program at the point it was interrupted.

Hardware controlled I/O

- To increase the speed of data transfer between processors memory and I/O, the hardware controlled I/O is used. It is commonly referred to as Direct Memory Access (DMA). The hardware which controls this data transfer is commonly known as DMA controller.
- The DMA controller sends a HOLD signal to the processor to initiate data transfer. In response to HOLD signal, processor releases its data, address and control buses to the DMA controller. Then the data transfer is controlled at high speed by the DMA controller without the intervention of the processor.

- After data transfer, DMA controller sends low on the HOLD pin, which gives the control of data, address, and control buses back to the processor.
- This type of data transfer is used for large data transfers.

I/O Control by handshake signals

- The handshake signals are used to ensure the readiness of the I/O device and to synchronize the timing of the data transfer. In this data transfer, the status of handshaking signals are checked between the processor and an I/O device and when both are ready, the actual data is transferred.

Parallel and Serial Interface

- An I/O interface consists of circuits which connect an, I/O device to a computer bus.
- As shown in Fig. 8.9.1 on one side of the interface we have the bus signals for address, data and control. On the other side we have a data path with its associated controls to transfer data between the interface and the I/O device.

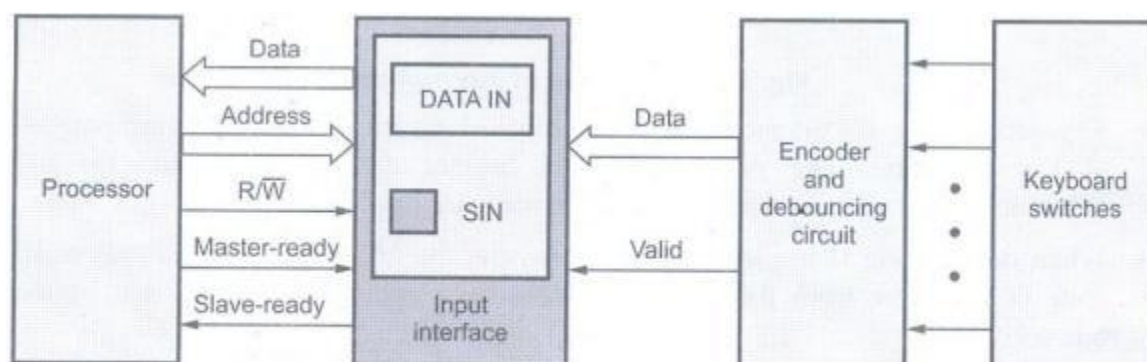


Fig. 8.9.1 Hardware components for connecting a keyboard to a processor

- The interface can be classified as serial interface or parallel interface.

Parallel Interface

- Parallel interface is used to send or receive data having group of bits (8-bits or 16-bits) simultaneously.
- According to usage, hardware and control signal requirements parallel interfaces are classified as input interface and output interface.
- Input interfaces are used to receive the data whereas output interfaces are used to send the data.

Input Interface

- Commonly used input device is a keyboard. Fig. 8.9.1 shows the hardware components needed for connecting a keyboard to a processor.
- A typical keyboard consists of mechanical switches that are normally open. When key is pressed, corresponding signal alters and encoder circuit generates ASCII code for the corresponding key.
- For interfacing switches to the microprocessor based systems, usually push button keys are used. These push button keys when pressed, bounces a few times, closing and opening the contacts before providing a steady reading, as shown in the Fig. 8.9.2. Reading taken during bouncing period may be faulty. Therefore, microprocessor must wait until the key reach to a steady state; this is known as key debounce.

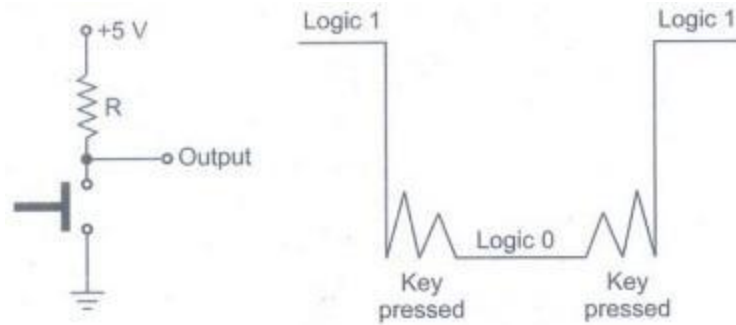


Fig. 8.9.2 Bouncing of key switch

- Key-debouncing circuit included in the block diagram of Fig. 8.9.3 eliminate effect of key bouncing. The problem of key bounce can be eliminated using key debounce circuit (hardware approach) or software approach.
- When debouncing is implemented in software, the I/O routine that reads a ASCII code of character from the keyboard waits long enough to ensure that bouncing has subsided.
- Fig. 8.9.3 shows the hardware approach to prevent key bouncing. It consists of flip-flop. The output of flip-flop shown in Fig. 8.9.3 is logic 1 when key is at position A (unpressed) and it is logic 0 when key is at position B, as shown in Table 8.9.1. It is important to note that, when key is in between A and B, output does not change, preventing bouncing of key output. In other words we can say that output does not change during transition period, eliminating key debouncing.

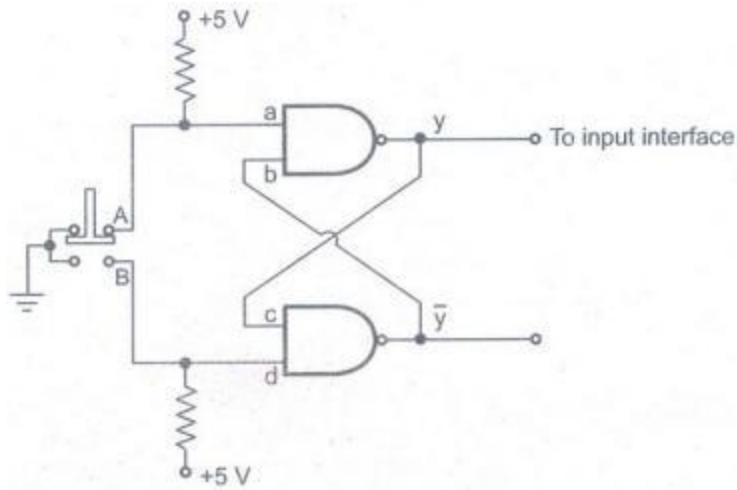


Fig. 8.9.3

key position	a	b	Y	c	d	$\bar{Y}$
A	0	0	1	1	1	0
B	1	1	0	0	0	1
Between A and B	1	$\bar{Y}$	No change	Y	1	No change

Table 8.9.1

- The output of encoder in Fig. 8.9.4 consists of the bits that represent the encoded character and one control signal called valid, which indicates that a key is being pressed. The encoder circuit sends this information to the interface circuit.

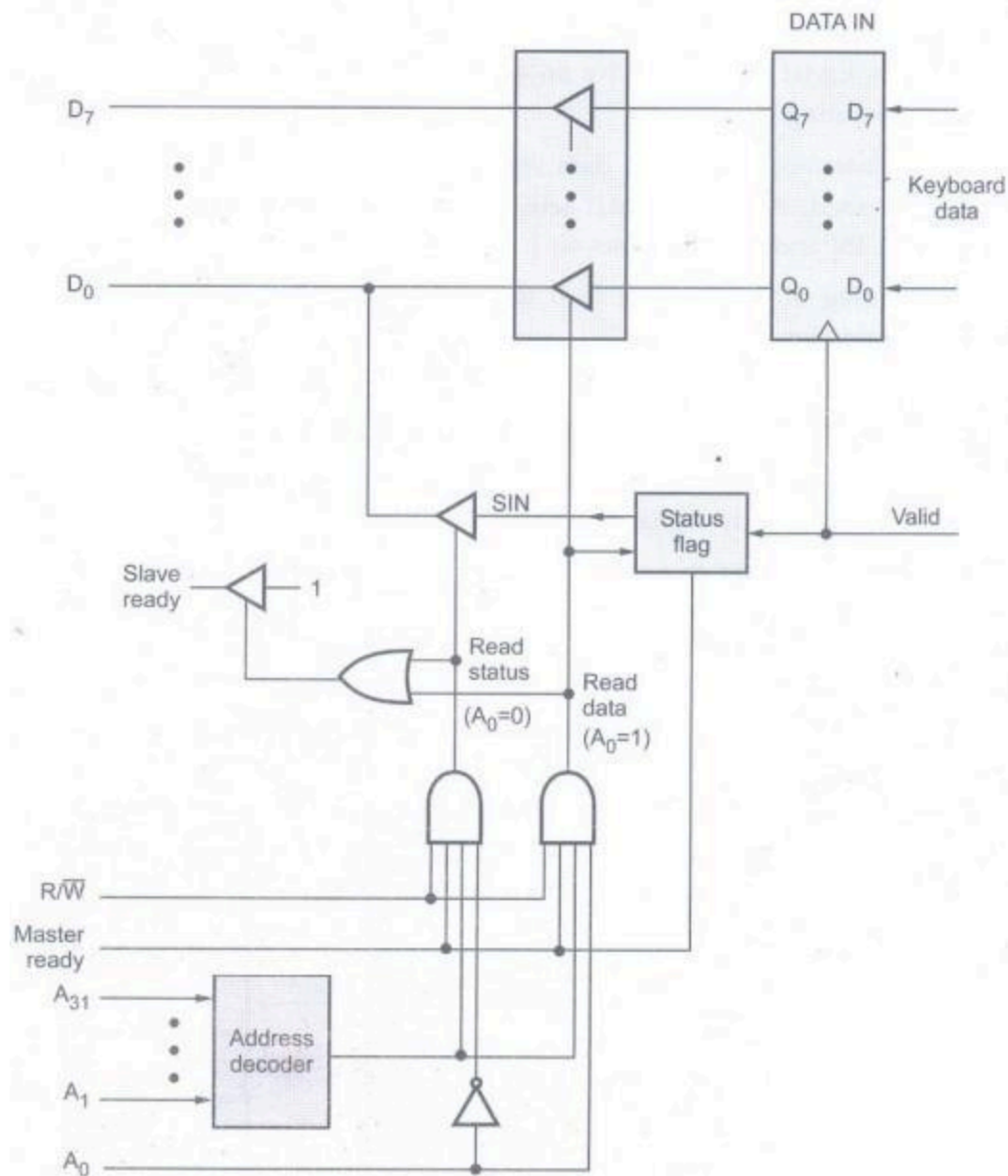


Fig. 8.9.4 Input interface circuit

- The interface circuit consists of a data register, DATA IN, and a status flag, SIN.
- When key is pressed, the valid signal is activated, (changes from 0 to 1) causing the ASCII code to be loaded into DATA IN and SIN to be set to 1. The status flag SIN is cleared to 0 when the processor reads the contents of the DATA IN register.

- The interface circuit is connected to an asynchronous bus on which transfers are controlled by using the handshake signals master-ready and slave-ready.
- The Fig. 8.9.4 shows the typical internal circuitry for input interface. Here, we receive the data input from the keyboard input device.
- When the key is pressed, its switch closes and establishes a path for an electrical signal. This signal is detected by an encoder circuit that generates ASCII code for the corresponding character.
- The input interface consists of data register, DATA IN, and status flag, SIN. When a key is pressed, the valid signal activates and causes the ASCII code to be loaded into DATA IN and SIN to be set to 1.
- The status flag SIN is cleared to 0 when the processor reads the contents of the DATA IN register.
- An address decoder is used to select the input interface when the high-order 31 bits of an address corresponds to any of the addresses assigned to this interface.
- Address bit determines whether the status or the data register is to be read when the Master-ready signal is active.
- The control handshake is accomplished by activating the slave-ready signal when either Read-status or Read-data is equal to 1.

Output Interface

- The Fig. 8.9.5 shows typical example of output interface which is used to interface parallel printer.

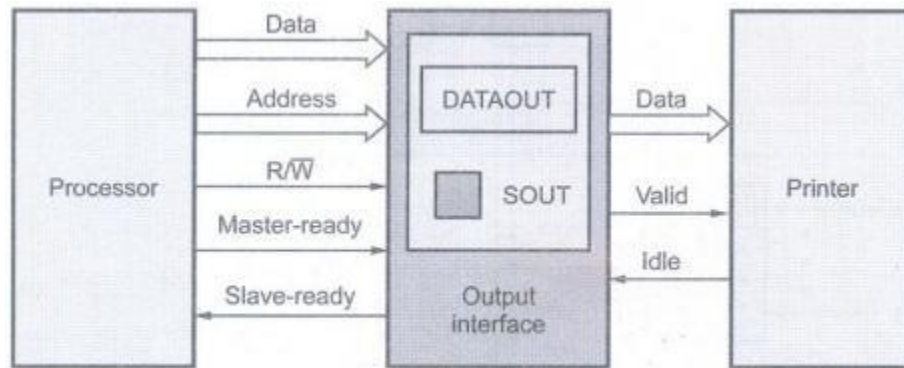


Fig. 8.9.5 Printer and processor interface

- The output interface contains a data register, DATAOUT, and a status flag, SOUT.
- The SOUT flag is set to 1 when the printer is ready to accept another character, and it is cleared to 0 when a new character is loaded into DATAOUT by the processor.
- The Fig. 8.9.6 shows the detail internal circuit for output interface.

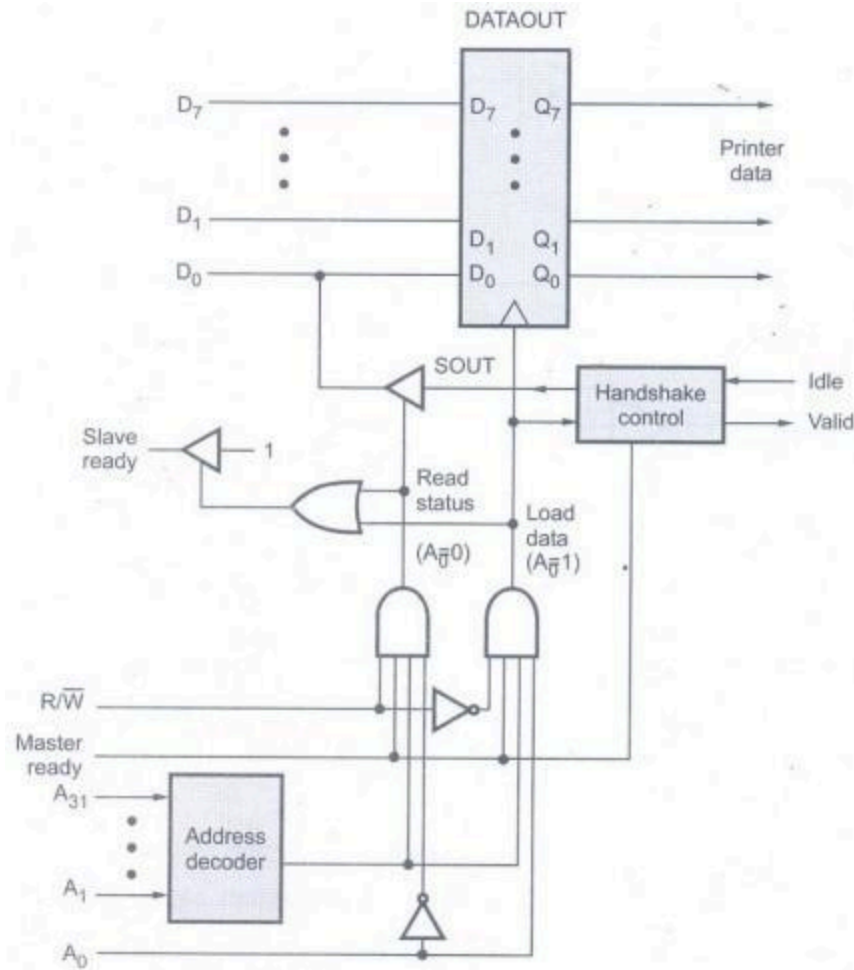


Fig. 8.9.6 Output interface circuit

Combined Input/Output Interface

- Combine input and output interfaces is shown in the Fig. 8.9.7.
- The 30-bits of higher order address ($A_{31} - A_2$) are used to select overall interface. The low-order two bits of address ($A_0 - A_1$) are used to select one of the three addressable locations in the interface. These are: Two data registers and one status register.

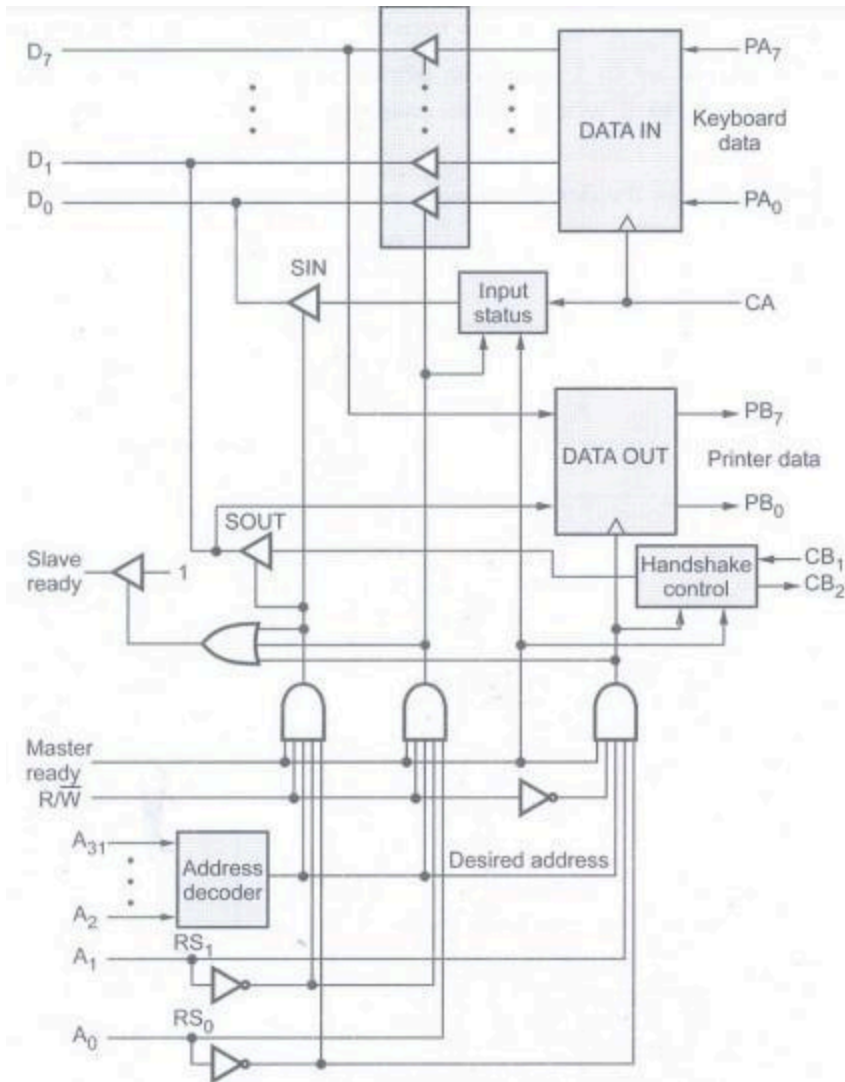


Fig. 8.9.7 Combined input/output interface circuit

- Flags SIN and SOUT are in the status register.
- Labels RS_1 and S_0 used for inputs that determines the selection of desired register.

8.9.1.4 Programmable Parallel Interface

- The input and output interfaces can be combined into a single interface and the direction of data flow can be controlled by data direction register.

- Single interface can be programmed to use for input the data or output the data. Such a interface is known as programmable parallel interface.
- The Fig. 8.9.8 shows the simplified block diagram of 8-bit programmable parallel interface.

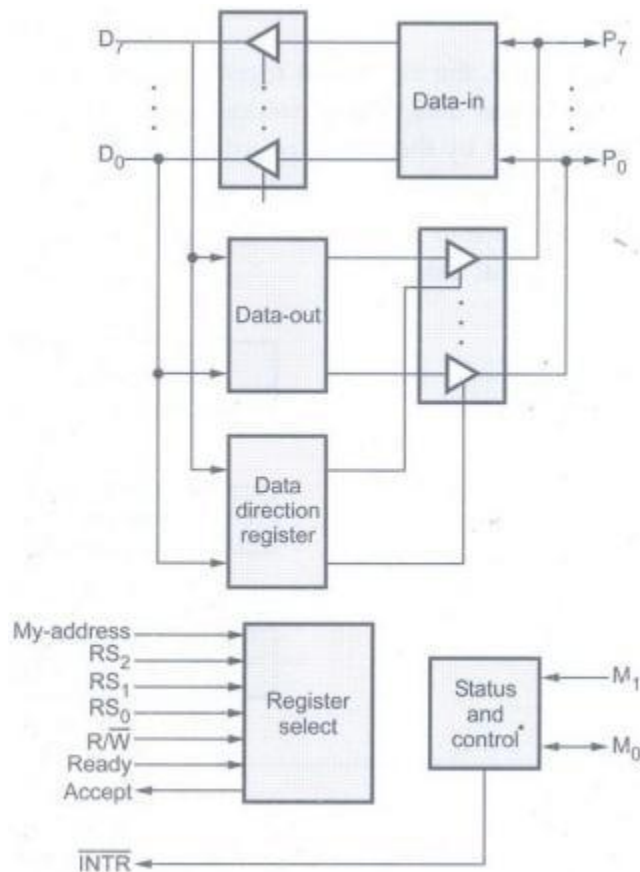


Fig. 8.9.8

- Data and interface lines are bidirectional. Their direction is controlled by the Data Direction Register (DDR).
- Two lines M_0 and M_1 are connected to the status and control. These two lines decide the mode of operation of the parallel interface, i.e. whether to operate the parallel interface as a simple input/output or handshake input/output.

- Ready and Accept signals are provided as handshaking signals.
- The $\overline{\text{INTR}}$ signal is also provided to allow interrupt drives I/O data transfer.

Serial Interface

- A serial interface is used to transmit / receive data serially, i.e., one bit at a time.
- A key feature of an interface circuit for a serial port is that it is capable of communicating in a bit serial fashion on the device side and in a bit parallel fashion on the processor side.
- A shift register is used to transform information between the parallel and serial formats. The Fig. 8.9.9 shows the block diagram of typical internal circuit for serial interface.
- As shown in the Fig. 8.9.9, the input shift register accepts serial data bit by bit and converts it into the parallel data. The converted parallel data is loaded in the data register and it is then read by the processor using data bus.
- When it is necessary to send data serially, the data is loaded into DATAOUT register. It is then loaded into output shift register. Output shift register converts this parallel data into serial data.

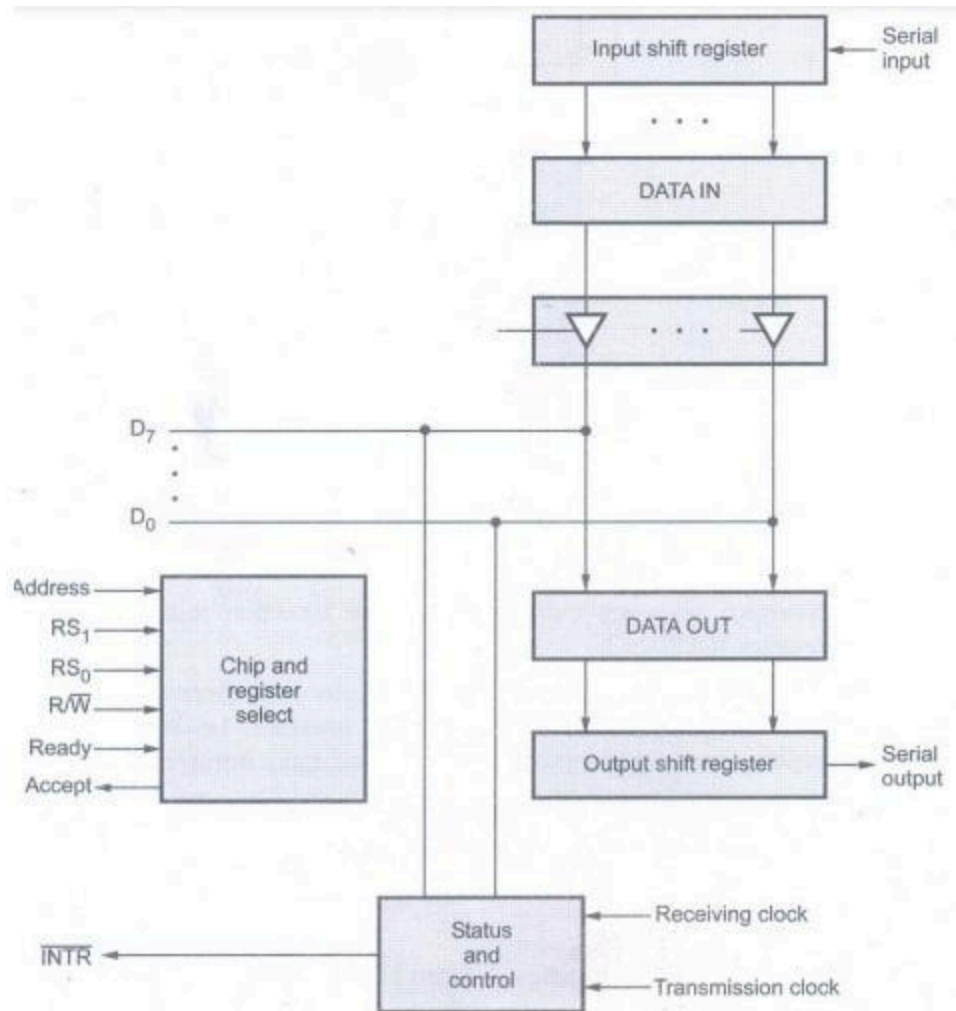


Fig. 8.9.9 A serial interface

Comparison between Serial and Parallel Interface

- In parallel interface number of lines required to transfer data depend on the number of bits to be transferred.
- For transmitting data over a long distance, using parallel interface is impractical due to the increase in cost of cabling.
- Parallel interface is also not practical for devices such as cassette tapes or a CRT terminal. In such situations, serial interface is used.

- In serial interface one bit is transferred at a time over a single line.

Sr. No.	Serial interface	Parallel interface
1.	It transfer data one bit at a time.	It can transmit more than one data bit at a time.
2.	Lower data transfer rate.	Faster data transfer rate.
3.	Needs less number of wires to connect devices in the system.	Needs more number of wires to connect devices in the system.
4.	Well suited for long distances, because fewer wires are used as compared to a parallel bus.	The interconnection penalty increases as distances increase. Thus not suitable for long distance interface.

Table 8.9.2 Comparison between serial and parallel interface

Interrupt I/O

AU: Dec.-06, 07, 08, 09, 10, 11, 12, 16, 18, May-06, 07, 08, 09, 12, 13

- Sometimes it is necessary to have the computer automatically execute one of a collection of special routines whenever certain conditions exists within a program or the computer system e.g. It is necessary that computer system should give response to devices such as keyboard, sensor and other components when they request for service.

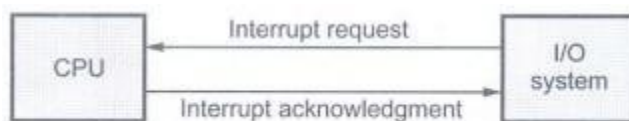


Fig. 8.8.1

- This method provides an external asynchronous input that would inform the processor that it should complete whatever instruction that is currently being executed and fetch a new routine (Interrupt Service Routine) that will service the

requesting device. Once this servicing is completed, the processor would resume exactly where it left off. The event that causes the interruption is called interrupt and the special routine executed to service the interrupt is called Interrupt Service Routine (ISR).

- The interrupt service routine is different from subroutine because the address of ISR is predefined or it is available in Interrupt Vector Table (IVT), whereas subroutine address is necessarily to be given in subroutine CALL instruction. IRET instruction is used to return from the ISR whereas RET instruction is used to return from subroutine. IRET instruction restores flag contents along with CS and IP in the IA-32 architecture; however RET instruction only restores CS and IP contents.
- An interrupt caused by an external signal is referred as a hardware interrupt.
- Conditional interrupts or interrupts caused by special instructions are called software interrupts.

Enabling and disabling interrupts

- Most of the processors provide the masking facility. In the processor those interrupts which can be masked under software control are called maskable interrupts.
- The interrupts which can not be masked under software control are called non-maskable interrupts.
- Maskable interrupts are enabled and disabled under program control. By setting or resetting particular flip-flops in the processor, interrupts can be masked or unmasked, respectively.

- When masked, processor does not respond to the interrupt even though the interrupt is activated.

Vectored interrupts

- When the external device interrupts the processor (interrupt request), processor has to execute interrupt service routine for servicing that interrupt. If the internal control circuit of the processor produces a CALL to a predetermined memory location which is the starting address of interrupt service routine, then that address is called vector address and such interrupts are called vector interrupts. For vector interrupts fastest and most flexible response is obtained since such an interrupt causes a direct hardware-implemented transition to the correct interrupt-handling program. This technique is called vectoring. When processor is interrupted, it reads the vector address and loads it into the PC.

Interrupt nesting

- For some devices, a long delay in responding to an interrupt request may cause error in the operation of computer. Such interrupts are acknowledged and serviced even though processor is executing an interrupt service routine for another device.
- A system of interrupts that allows an interrupt service routine to be interrupted is known as nested interrupts.

Interrupt priority

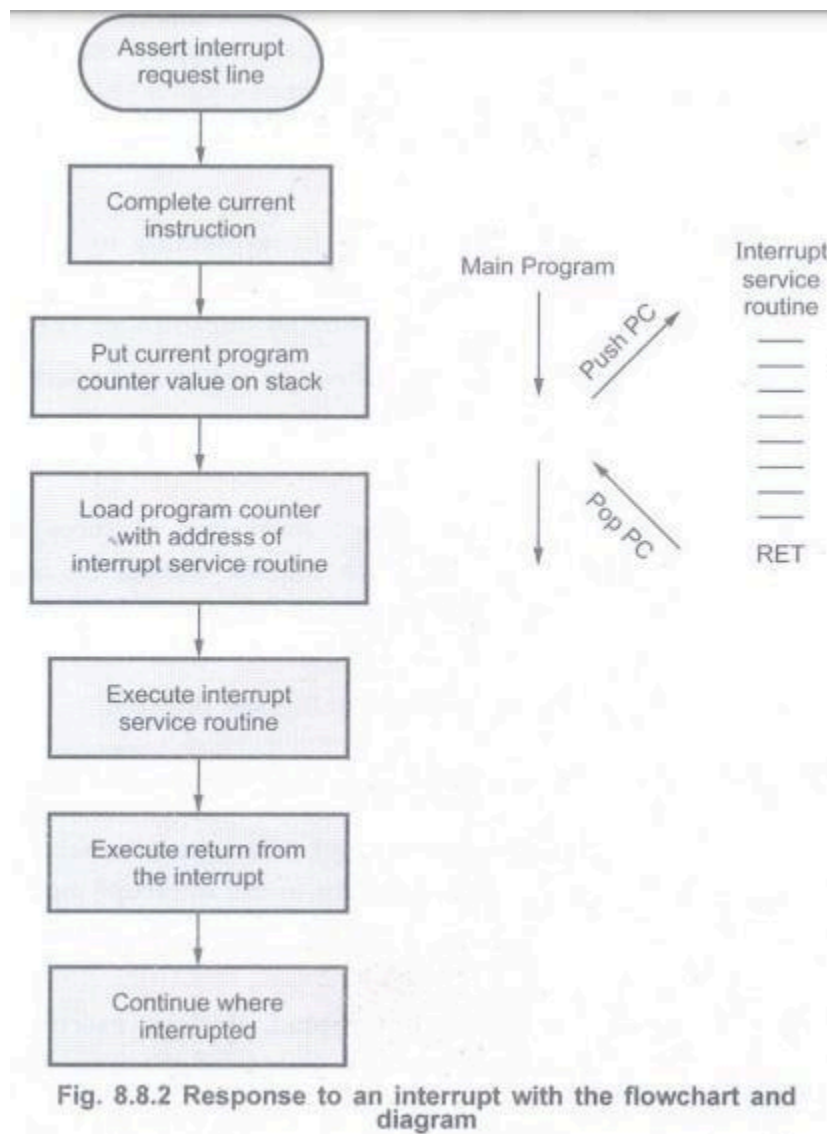
- When interrupt requests arrive from two or more devices simultaneously, the processor has to decide which request should be serviced first and which one should be delayed. The processor takes the decision with the help of interrupt priorities.

- It accepts the request having the highest priority.

Recognition of Interrupt and Response to Interrupt

- The CPU recognizes the interrupt when the external asynchronous input (interrupt input) is asserted (a signal is sent to the interrupt input) by an I/O device.
 - In response to an interrupt a special sequence of actions are performed. These are as follows:
 - When a processor is interrupted, it stops executing its current program and calls a special routine which "services" the interrupt. The event that causes the interruption is called interrupt and the special routine which is executed is called interrupt service routine.
1. The processor completes its current instruction. No instruction is cut-off in the middle of its execution.
 2. The program counter's current contents are stored on the stack. Remember, during the execution of an instruction the program counter is pointing to the memory location for the next instruction.
 3. The program counter is loaded with the address of an interrupt service routine.
 4. Program execution continues with the instruction taken from the memory location pointed by the new program counter contents.
 5. The interrupt program continues to execute until a return instruction is executed.
 6. After execution of from the RET instruction processor gets the old address (the address of the next instruction where the interrupt service routine was called.) of

the program counter from the stack and puts it back into the program counter. This allows the interrupted program to continue executing at the instruction following the one where it was interrupted. Fig. 8.8.2 shows the response to interrupt with flowchart and diagram.



Comparison between Programmed I/O and Interrupt Driven I/O

The Table 8.8.1 gives the comparison between programmed I/O and interrupt driven I/O.

Sr. No.	Programmed I/O	Interrupt driven I/O
1.	In programmed I/O, processor has to check each I/O device in sequence and in effect 'ask' each one if it needs communication with the processor. This checking is achieved by continuous polling cycle and hence processor can not execute other instructions in sequence.	External asynchronous input is used to tell the processor that I/O device needs its service and hence processor does not have to check whether I/O device needs it service or not.
2.	During polling processor is busy and therefore, have serious and decremental effect on system throughput.	In interrupt driven I/O, the processor is allowed to execute its instructions in sequence and only stop to service I/O device when it is told to do so by the device itself. This increases system throughput.
3.	It is implemented without interrupt hardware support.	It is implemented using interrupt hardware support.
4.	It does not depend on interrupt status.	Interrupt must be enabled to process interrupt driven I/O.
5.	It does not need initialization of stack.	It needs initialization of stack.
6.	System throughput decreases as number of I/O devices connected in the system increases.	System throughput does not depend on number of I/O devices connected in the system.

Table 8.8.1 Comparison between programmed I/O and interrupt driven I/O

Interrupt Priority Schemes

•In case polling to identify the interrupting device, priority is automatically assigned by the order in which devices are polled. Therefore, no further arrangement is required to accommodate simultaneous interrupt requests. However, in case of vectored interrupts, the priority of any device is usually determined by the way the device is connected to the processor. Most common way to connect the devices is to form a daisy chain, as shown in the Fig. 8.8.3. As shown in the Fig. 8.8.3, the interrupt request line (**INTR**) is common to all devices and the interrupt acknowledge line (INTA) is connected in a daisy-chain fashion. In daisy-chain fashion the signal is allowed to propagate serially through the devices. When more than one devices issue an interrupt request, the (**INTR**) line is activated

and processor responds by setting the INTA line. This signal is received by device 1. Device 1 passes the signal to the device 2 only if it does require any service. If device 1 requires service, it blocks the INTA line and puts its identification code on the data lines. Therefore, in daisy-chain arrangement, the device that is electrically closest to the processor has the highest priority.

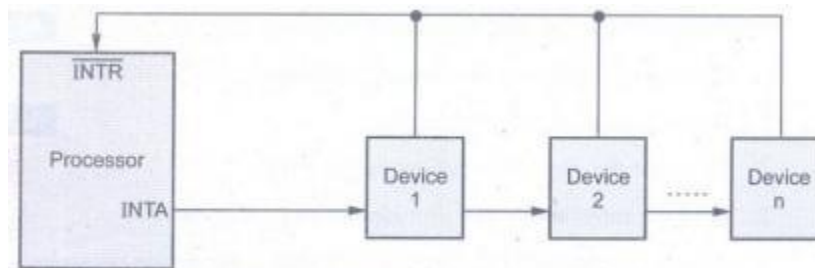


Fig. 8.8.3 Interrupt priority system using daisy-chain

- The Fig. 8.8.4 shows another arrangement for handling priority interrupts. Here, device are organised in groups, and each group is connected at a different priority level. Within a group, devices are connected in a daisy-chain.

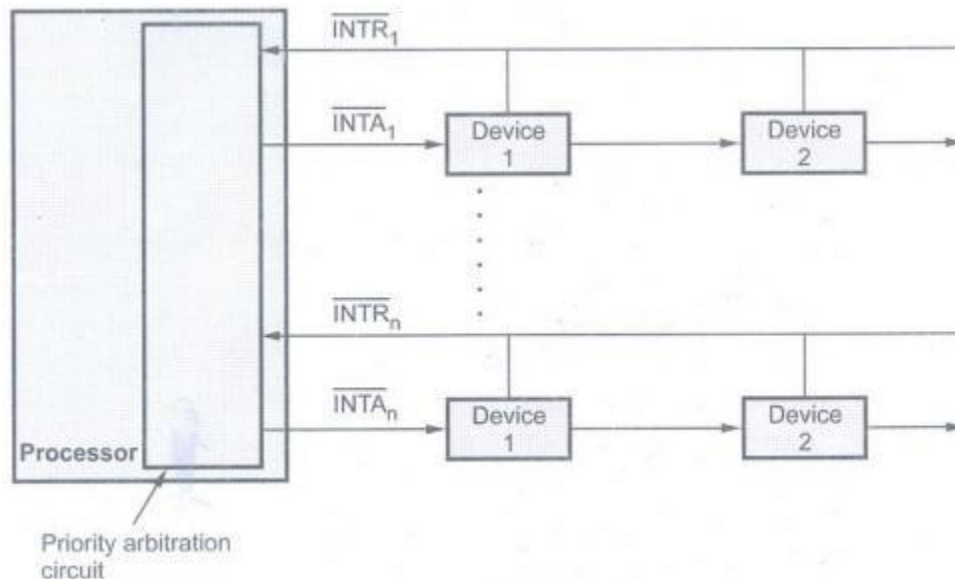


Fig. 8.8.4 Arrangement of priority groups